

3D Surface Reconstruction via the Photometric Stereo Method

JC Vaught and Ty Dangerfield
Department of Mechanical Engineering, University of South Carolina

December 8, 2025

Abstract

We present an analytical and numerical validation of the photometric stereo method for 3D surface reconstruction with applications to industrial defect detection. This paper details seven primary contributions: (1) a complete mathematical framework including Lambertian reflectance, Euler–Lagrange derivation of the Poisson equation, and separation-of-variables analytical solutions; (2) four distinct Poisson solvers—FFT-based (periodic), DCT-based (Neumann), finite difference (Dirichlet), and conjugate gradient iterative (Dirichlet)—with detailed algorithmic descriptions; (3) experimental validation across eight diverse test surfaces spanning smooth, sharp-edged, and oscillating geometries; (4) comprehensive ablation studies on light count, noise robustness, and Tikhonov regularization; (5) systematic solver comparison using identical divergence fields with 32 uniformly distributed light sources; (6) dual implementations in both Python and MATLAB with complete code listings; and (7) quantitative analysis demonstrating reconstruction RMSE ranging from 0.0001 (sinusoid with CG solver) to 0.1016 (saddle), with the CG-Iterative solver achieving the best or tied-best results on seven of eight test surfaces.

Table 1: Individual Contributions

Contribution	Primary Contributor(s)
MATLAB implementation of complete program	Ty Dangerfield
Python implementation of complete program	JC Vaught
Photometric principle derivation	Ty Dangerfield & JC Vaught
Gradient integration with Poisson solver	Ty Dangerfield
Separation of variables and numerical derivations	JC Vaught
Report chapters 1, 3, and 4 (initial)	JC Vaught
Report chapters 1, 2, and 5 (initial)	Ty Dangerfield
Report restructuring and enhanced documentation	JC Vaught & Ty Dangerfield
Code architecture refinement and expansion	JC Vaught & Ty Dangerfield

Table 2: Meeting Participation

Date	JC Vaught	Ty Dangerfield
19 Nov 2025	Present	Present
20 Nov 2025	Present	Present
07 Dec 2025	Present	Present

Contents

1	Introduction	5
1.1	What We Want to Model	5
1.2	Why This Problem is Important	5
1.3	Comparison with Alternative Methods	6
1.4	The Core PDE Problem	6
2	Mathematical Foundation	7
2.1	Photometric Stereo Principle	7
2.1.1	Image Formation Model	7
2.1.2	Lambertian Reflectance	8
2.1.3	The Linear System Formulation	8
2.1.4	Least-Squares Solution via Pseudoinverse	9
2.1.5	Conditioning and Noise Sensitivity	10
2.1.6	Shadow Handling and Robust Estimation	10
2.2	Gradient Integration via Poisson Equation	11
2.2.1	Variational Formulation	11
2.2.2	Euler–Lagrange Derivation	11
2.3	Boundary Condition Theory	12
2.3.1	Dirichlet Boundary Conditions	12
2.3.2	Neumann Boundary Conditions	12
2.3.3	Periodic Boundary Conditions	12
2.3.4	Compatibility and Uniqueness	13
2.4	Analytical Solution via Separation of Variables	13
2.4.1	Eigenfunction Expansion	14
3	Numerical Methods	15
3.1	From Continuous PDE to Discrete Grid	15
3.1.1	Approximating Derivatives (Taylor Series)	15
3.1.2	The 5-Point Stencil Pattern	15
3.1.3	Building the Linear System	16
3.2	Solver 1: FFT-Based (Spectral Method)	17
3.2.1	Mathematical Approach	17
3.2.2	Boundary Condition: Periodic	18
3.2.3	Algorithm Steps	18
3.3	Solver 2: Finite Difference (Dirichlet Boundaries)	19
3.3.1	Mathematical Approach	19
3.3.2	Boundary Condition: Fixed Values	19
3.3.3	Algorithm Steps	20
3.4	Solver 3: DCT-Based (Neumann Boundaries)	21
3.4.1	Mathematical Formulation	21
3.4.2	Boundary Condition: Zero Flux	21
3.4.3	Algorithm	22
3.5	Solver 4: CG-Iterative (Dirichlet Boundaries)	23
3.5.1	Mathematical Formulation	23
3.5.2	Dirichlet Boundary Conditions	23

3.5.3	Conjugate Gradient Iteration	23
3.5.4	Algorithm	24
3.6	Regularization (Noise Handling)	25
3.6.1	Why Regularization is Needed	25
3.6.2	Tikhonov Derivation	25
3.6.3	Choosing the Parameter	26
3.6.4	L-Curve Method	26
3.6.5	Discrepancy Principle	26
3.6.6	Cross-Validation	27
4	Implementation	28
4.1	Photometric Stereo Pipeline	28
4.1.1	Image Acquisition and Preprocessing	28
4.1.2	Light Matrix Construction	28
4.1.3	Per-Pixel Normal Estimation	29
4.1.4	Gradient Field Computation	29
4.2	Poisson Solver Integration	30
4.2.1	Divergence Field Construction	30
4.2.2	Solver Selection Logic	30
4.2.3	Post-Processing (Mean Centering)	31
5	Software Architecture	32
5.1	Package Structure	32
5.2	Core Modules	33
5.2.1	Surface Generation (<code>surfaces/</code>)	33
5.2.2	Photometric Stereo (<code>photometric/</code>)	33
5.2.3	Poisson Solvers (<code>solvers/</code>)	33
5.3	Running Experiments	33
6	Experimental Validation	35
6.1	Test Surfaces	35
6.1.1	Surface Selection Rationale	35
6.2	Experimental Protocol	36
6.2.1	Common Setup Parameters	36
6.2.2	Evaluation Metrics	37
6.3	Core Validation	37
6.3.1	Validation 1: Poisson Solver Validation (Gradient $\rightarrow$ Height)	38
6.3.2	Validation 2: Full Photometric Stereo Pipeline (Image $\rightarrow$ Height)	38
6.4	Smooth Curved Surface Experiments	39
6.5	Sharp-Edged Surface Experiments	39
6.6	Oscillating Surface Experiments	39
6.7	Ablation Studies	40
6.7.1	Light Count Sweep ($m = 3$ to 20)	40
6.7.2	Noise Robustness ($\sigma = 0$ to 0.08)	40
6.7.3	Resolution Convergence Study	41
6.8	Solver Comparison Experiments	41
6.8.1	Protocol: Same Divergence Field, Different Solvers	41
6.8.2	FFT Solver (Periodic BC)	42

6.8.3	Finite Difference Solver (Dirichlet BC)	42
6.8.4	DCT Solver (Neumann BC)	42
7	Results and Discussion	44
7.1	Quantitative Results Summary	44
7.1.1	Key Observations	44
7.2	Smooth Curved Surface Results	44
7.2.1	Gaussian Surface	45
7.2.2	Sphere Surface	45
7.2.3	Ellipsoid Surface	46
7.3	Sharp-Edged Surface Results	47
7.3.1	Cone Surface	47
7.3.2	Cube Surface	48
7.4	Oscillating Surface Results	49
7.4.1	Saddle Surface	49
7.4.2	Sinusoid Surface	50
7.4.3	MATLAB Peaks Surface	51
7.5	Solver Comparison Analysis	52
7.5.1	Boundary Condition Impact	52
7.5.2	Normal Map Quality	53
7.6	Ablation Study Results	53
7.6.1	Light Count Sweep	53
7.6.2	Noise Robustness	53
7.6.3	Tikhonov Regularization	54
7.7	Discussion	54
7.7.1	Key Findings	54
7.7.2	Limitations	54
A	Python Code Listings	55
A.1	Configuration (config.py)	55
A.2	FFT Solver (solvers/fft_periodic.py)	55
A.3	DCT Solver (solvers/dct_neumann.py)	55
A.4	CG Solver (solvers/cg_iterative.py)	56
A.5	FD Solver (solvers/fd_dirichlet.py)	57
A.6	Visualization (visualization/)	57
A.7	Surface Generators (surfaces/)	58
A.8	Photometric Stereo Pipeline	58
A.9	Main Runner (runner.py)	59
B	MATLAB Code Listings	60
B.1	Configuration (config.m)	60
B.2	Main Runner (runner.m)	60
B.3	Surface Generators (surfaces/)	60
B.4	FFT Solver (solve_poisson_fft.m)	61
B.5	DCT Solver (solve_poisson_dct.m)	61
B.6	FD/CG Solver (solve_poisson_fd.m)	61
B.7	Photometric Stereo Functions	62
B.8	Visualization (visualization/save_surface_figures.m)	63

1 Introduction

1.1 What We Want to Model

Three-dimensional surface reconstruction is a fundamental problem in manufacturing applications and mechanical engineering more broadly. Photometric stereo is one possible solution among many, yielding highly accurate surface reconstruction compared to a dual-camera setup, which is more often used in robotic applications.

Given multiple 2D images of an object illuminated from different directions, we derive the PDE, analytical solution, and numerical solution that create an accurate reconstruction of the 3D depth map or surface geometry.

1.2 Why This Problem is Important

Photometric stereo has impact across multiple domains summarized in Table 3, including industrial inspection, where it supports surface defect detection and tolerance verification; medical imaging, where it enables high-fidelity reconstructions of anatomical surfaces for planning; archaeology, where non-destructive digitization preserves fragile artifacts; robotics and autonomous systems, where detailed surface geometry informs grasping and navigation; and reverse engineering, where recovered shapes seed CAD models for subsequent design and analysis.

Table 3: Representative Application Domains for Photometric Stereo

Domain	Example Use
Industrial inspection	Surface defect detection, geometric tolerance checks
Medical imaging	Preoperative surface reconstruction for planning
Archaeology	Non-destructive digitization of artifacts
Robotics	Shape-aware grasp planning and manipulation
Autonomous vehicles	High-fidelity surface maps for navigation
Reverse engineering	Recovering geometry for CAD modeling

Photometric stereo is an ideal solution for many surface extraction tasks owing to the reduction in moving parts. For example, in structure-from-motion methods, the movement of the camera must be precisely monitored and controlled, necessitating careful calibration; even minor calibration errors can severely impact reconstruction quality. Stereo cameras avoid motion but still suffer from calibration issues because the distance, angle, and focal length of each camera—the intrinsic parameters—must be known precisely to estimate depth. Any error there can distort the recovered geometry due to the trigonometric basis of the method. Photometric stereo, on the other hand, is a completely solid-state approach with a single camera, operates at a higher rate than structure-from-motion, and has lower data requirements than stereo systems. Because it leverages illumination direction to infer depth from shading and then reconstruct normals, it is inherently more tolerant to noise and avoids multi-camera calibration drift.

However, photometric stereo does suffer from one critical constraint: the illumination sequence must be precisely controlled. As a result, it cannot be used effectively in uncontrolled environments such as UAVs or general-purpose robotics, which is why stereo cameras remain the de facto standard outside manufacturing despite their calibration sensitivity. Nevertheless, the fundamental challenge in photometric stereo still lies in *integrating* noisy normal estimates into a globally consistent 3D surface, which requires solving a partial differential equation: the **Poisson equation**.

1.3 Comparison with Alternative Methods

Table 4 compares photometric stereo with other common 3D reconstruction techniques across key performance dimensions. The comparison highlights why photometric stereo is particularly well-suited for controlled industrial environments despite requiring specialized lighting infrastructure.

Table 4: Comparison of 3D Reconstruction Methods

Method	Advantages	Disadvantages	Typical Use Cases
Photometric Stereo	Single camera; high spatial resolution; dense surface normals; solid-state (no moving parts)	Requires controlled lighting; assumes Lambertian reflectance; limited to near-field	Defect detection, material inspection, quality control
Stereo Vision	Passive (no special lighting); works outdoors; real-time capable	Requires precise calibration; texture-dependent; ambiguity in textureless regions	Robotics, autonomous vehicles, SLAM
Structure from Motion	Single camera; scales to large scenes; robust to illumination changes	Requires camera motion; computationally intensive; drift accumulation	3D scanning, mapping, archaeological digitization
Structured Light	High accuracy; active illumination; fast acquisition	Sensitive to ambient light; limited range; projector-camera calibration required	Industrial metrology, 3D printing, facial scanning
Time-of-Flight	Direct depth measurement; real-time; works in low texture	Lower resolution; sensitive to multipath reflections; expensive hardware	Gesture recognition, indoor navigation, human-computer interaction

Photometric stereo excels in industrial settings where lighting can be tightly controlled and surface finish is relatively uniform. The method’s ability to capture dense normal fields from a single viewpoint makes it ideal for in-line inspection systems where throughput and repeatability are critical. By contrast, methods like stereo vision or time-of-flight are better suited to unstructured environments (e.g., outdoor robotics) where lighting variability and scene dynamics preclude the use of synchronized illumination sequences.

1.4 The Core PDE Problem

The fundamental problem reduces to solving the 2D Poisson equation in a rectangular domain:

$$\nabla^2 z(x, y) = f(x, y), \quad (x, y) \in \Omega = [0, L_x] \times [0, L_y] \quad (1)$$

where

$z(x, y)$ = unknown surface height over the image domain

$$f(x, y) = \frac{\partial p}{\partial x} + \frac{\partial q}{\partial y} \quad (\text{divergence of Poisson-derived gradient estimates})$$

$$\Omega = [0, L_x] \times [0, L_y] \quad (\text{rectangular image domain})$$

This project demonstrates both analytical (separation of variables) and numerical (FFT-based and finite-difference) methods to solve this fundamental PDE and validate the solutions on synthetic 3D surfaces.

2 Mathematical Foundation

This chapter establishes the theoretical foundations for photometric stereo reconstruction. We begin with the photometric stereo principle, deriving the relationship between observed image intensities and surface normals under Lambertian reflectance. We then formulate the gradient integration problem as a Poisson equation, providing both variational and Euler-Lagrange derivations. Next, we develop the theory of boundary conditions—Dirichlet, Neumann, and periodic—and their impact on solution uniqueness. Finally, we present an analytical solution via separation of variables, including eigenfunction expansions and worked examples that validate our numerical methods.

2.1 Photometric Stereo Principle

Photometric stereo recovers surface normals from multiple images of a static scene illuminated by different light sources. The key insight is that shading variations across images encode the local surface orientation. Table 5 summarizes the key modeling assumptions underlying this approach.

Table 5: Photometric Stereo Modeling Assumptions

Assumption	Description
Lambertian reflectance	Surface reflects light diffusely in all directions; no specular highlights or glossy reflections
Orthographic projection	Camera uses parallel projection (unit focal length); no perspective distortion
Known light directions	Light source directions $\mathbf{L}_i$ are calibrated and known a priori
Static scene	Surface geometry and camera remain fixed across all images
Uniform albedo	In our derivations, albedo $\rho(x, y)$ is assumed constant, though the general formulation allows spatially varying albedo
No inter-reflections	Light reaches the surface directly without bouncing off other surfaces
Attached shadows only	Self-shadowing is handled via the max operator; cast shadows from other objects are not modeled

2.1.1 Image Formation Model

By assuming an ideal camera with a unit focal length, we map a 3D point $\mathbf{X}$ to image coordinates $\mathbf{x} = (x, y)$ for any arbitrary surface with depth $z(x, y)$. The measured image intensity I_i at pixel (x, y) under light source i is the product of three components: the camera calibration factor κ_i (encapsulating exposure, gain, and light source intensity), the material albedo $\rho(x, y) \in [0, 1]$ (the fraction of light reflected by the surface), and the geometric factor $\max(0, \mathbf{n}(x, y) \cdot \mathbf{L}_i)$ (the cosine of the angle between the surface normal $\mathbf{n}$ and light direction $\mathbf{L}_i$). These combine to yield the image formation equation:

$$I_i(x, y) = \kappa_i \rho(x, y) \max(0, \mathbf{n}(x, y) \cdot \mathbf{L}_i) \quad (2)$$

The max operator enforces attached shadows: when a light illuminates the surface from behind ($\mathbf{n} \cdot \mathbf{L}_i < 0$), the contribution is zero.

2.1.2 Lambertian Reflectance

The Lambertian assumption states that a surface appears equally bright from all viewing directions—light is reflected diffusely in all directions. Mathematically, the reflected radiance is proportional to $\cos \theta = \mathbf{n} \cdot \mathbf{L}$, where θ is the angle between the surface normal and the incident light direction.

To connect surface geometry to shading, we must express the surface normal in terms of the depth map $z(x, y)$. We begin by computing the tangent vectors to the surface. These tangent vectors represent the local orientation of the surface along the x and y coordinate directions:

$$\frac{\partial \mathbf{X}}{\partial x} = (1, 0, p), \quad \text{where } p = \frac{\partial z}{\partial x} \quad (3)$$

$$\frac{\partial \mathbf{X}}{\partial y} = (0, 1, q), \quad \text{where } q = \frac{\partial z}{\partial y} \quad (4)$$

Here, p and q represent the local slopes of the surface in the x and y directions, respectively. Note that the first two components of each tangent vector are $(1, 0)$ and $(0, 1)$ because we move one unit along the respective image coordinate while the surface height changes by p or q .

The surface normal is perpendicular to both tangent vectors and can be computed via their cross product:

$$\tilde{\mathbf{n}} = \frac{\partial \mathbf{X}}{\partial x} \times \frac{\partial \mathbf{X}}{\partial y} = (-p, -q, 1) \quad (5)$$

This unnormalized normal $\tilde{\mathbf{n}}$ points away from the surface but does not necessarily have unit length. The negative signs on p and q arise naturally from the cross product and ensure the normal points outward (away from the surface) rather than inward.

To obtain a unit normal vector (required for computing the geometric factor $\mathbf{n} \cdot \mathbf{L}$), we divide by the magnitude:

$$\mathbf{n} = \frac{\tilde{\mathbf{n}}}{\|\tilde{\mathbf{n}}\|} = \frac{(-p, -q, 1)}{\sqrt{p^2 + q^2 + 1}} \quad (6)$$

Equation (6) explicitly links the integrable gradient field (p, q) to the observed shading—this is the key relationship that enables gradient recovery from images.

2.1.3 The Linear System Formulation

Having established the relationship between surface normals and image intensities, we now formulate the inverse problem: given multiple images under different illumination, recover the surface normal at each pixel.

We introduce the scaled normal vector $\mathbf{g}(x, y) = \rho(x, y)\mathbf{n}(x, y)$, which combines the geometric information (normal direction) with the material property (albedo). This factorization is convenient because $\mathbf{g}$ appears linearly in the image formation equation.

Substituting the Lambertian reflectance model (6) into the image formation equation (2), we obtain for a single light source i :

$$I_i(x, y) = \kappa_i \mathbf{L}_i^T \mathbf{g}(x, y) \quad (7)$$

where we have absorbed the max operator into the formulation (assuming no self-shadowing for now; we address this later). This is a scalar equation linear in $\mathbf{g}$.

For m different light sources, we obtain m such equations at each pixel. Stacking these into a single matrix equation yields:

$$\underbrace{\begin{bmatrix} I_1(x, y) \\ I_2(x, y) \\ \vdots \\ I_m(x, y) \end{bmatrix}}_{\mathbf{I}(x, y) \in \mathbb{R}^m} = \underbrace{\begin{bmatrix} \kappa_1 \mathbf{L}_1^T \\ \kappa_2 \mathbf{L}_2^T \\ \vdots \\ \kappa_m \mathbf{L}_m^T \end{bmatrix}}_{S \in \mathbb{R}^{m \times 3}} \underbrace{\mathbf{g}(x, y)}_{\in \mathbb{R}^3} \quad (8)$$

Compactly, we write:

$$S\mathbf{g}(x, y) = \mathbf{I}(x, y) \quad (9)$$

The matrix S is called the *illumination matrix* and depends only on the known light directions and calibration factors. The vector $\mathbf{I}(x, y)$ contains the observed intensities, and our goal is to solve for $\mathbf{g}(x, y)$ at every pixel. For a unique solution to exist, S must have full column rank: $\text{rank}(S) = 3$. This requires at least three images ($m \geq 3$) with non-coplanar light directions. In practice, we use $m > 3$ to obtain an overdetermined system, which provides robustness against noise.

2.1.4 Least-Squares Solution via Pseudoinverse

When $m > 3$, the system (9) is overdetermined and generally has no exact solution due to measurement noise. We instead seek the least-squares solution that minimizes the residual:

$$\hat{\mathbf{g}}(x, y) = \arg \min_{\mathbf{g}} \|S\mathbf{g} - \mathbf{I}(x, y)\|_2^2 \quad (10)$$

Expanding the objective function:

$$\|S\mathbf{g} - \mathbf{I}\|_2^2 = (S\mathbf{g} - \mathbf{I})^T (S\mathbf{g} - \mathbf{I}) = \mathbf{g}^T S^T S \mathbf{g} - 2\mathbf{I}^T S \mathbf{g} + \mathbf{I}^T \mathbf{I} \quad (11)$$

Taking the gradient with respect to $\mathbf{g}$ and setting it to zero yields the *normal equations*:

$$\nabla_{\mathbf{g}} \|S\mathbf{g} - \mathbf{I}\|_2^2 = 2S^T S \mathbf{g} - 2S^T \mathbf{I} = 0 \quad \Rightarrow \quad S^T S \mathbf{g} = S^T \mathbf{I} \quad (12)$$

Assuming S has full column rank, $S^T S$ is invertible, and we obtain:

$$\mathbf{g}(x, y) = \underbrace{(S^T S)^{-1} S^T}_{S^+ \text{ (pseudoinverse)}} \mathbf{I}(x, y) \quad (13)$$

The matrix $S^+ = (S^T S)^{-1} S^T$ is the Moore–Penrose pseudoinverse of S . This solution minimizes the L^2 norm of the residual and is unique when $\text{rank}(S) = 3$. Once $\mathbf{g}(x, y)$ is computed, we recover the unit normal and albedo by exploiting the factorization $\mathbf{g} = \rho \mathbf{n}$:

$$\hat{\rho}(x, y) = \|\mathbf{g}(x, y)\|_2 \quad (14)$$

$$\hat{\mathbf{n}}(x, y) = \frac{\mathbf{g}(x, y)}{\|\mathbf{g}(x, y)\|_2} \quad (15)$$

The *direction* of $\mathbf{g}$ encodes the surface normal while its *magnitude* equals the albedo.

2.1.5 Conditioning and Noise Sensitivity

The accuracy of the recovered normals depends critically on the *condition number* of S :

$$\kappa(S) = \frac{\sigma_{\max}(S)}{\sigma_{\min}(S)} \quad (16)$$

where $\sigma_{\max}$ and $\sigma_{\min}$ are the largest and smallest singular values of S , respectively. To understand the impact of conditioning, consider the singular value decomposition (SVD) of S :

$$S = U\Sigma V^T \quad (17)$$

where $U \in \mathbb{R}^{m \times 3}$ and $V \in \mathbb{R}^{3 \times 3}$ are orthogonal matrices, and $\Sigma = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$ contains the singular values in descending order. The pseudoinverse can be expressed as:

$$S^+ = V\Sigma^{-1}U^T \quad (18)$$

If the measured intensities contain noise $\Delta \mathbf{I}$ such that $\|\Delta \mathbf{I}\|/\|\mathbf{I}\| = \epsilon$, the relative error in the recovered $\mathbf{g}$ is bounded by:

$$\frac{\|\Delta \mathbf{g}\|}{\|\mathbf{g}\|} \leq \kappa(S) \cdot \epsilon \quad (19)$$

This shows that the condition number $\kappa(S)$ controls the worst-case amplification of measurement noise. A large condition number indicates that small errors in $\mathbf{I}$ can lead to large errors in $\mathbf{g}$. To minimize noise sensitivity, we seek to minimize $\kappa(S)$, or equivalently, maximize $\sigma_{\min}(S)$. This is achieved when the light directions $\{\mathbf{L}_i\}$ are uniformly distributed over the unit sphere, ensuring that the columns of S span $\mathbb{R}^3$ evenly. In practice, lights are often arranged in a hemispherical configuration with uniform azimuthal and elevation spacing. Using $m > 3$ images provides two additional benefits: the least-squares solution averages out random noise, and the system becomes more robust to individual measurements corrupted by saturation or outliers.

2.1.6 Shadow Handling and Robust Estimation

The linear system (9) assumes that all lights illuminate the surface from the front ($\mathbf{n} \cdot \mathbf{L}_i > 0$). When a light source illuminates from behind, *self-shadowing* occurs, and the image formation equation becomes:

$$I_i(x, y) = 0 \quad \text{if } \mathbf{n}(x, y) \cdot \mathbf{L}_i < 0 \quad (20)$$

These shadowed measurements violate the linear model and must be excluded. The standard approach is to first identify light sources i for which $I_i(x, y) \approx 0$ (or below a threshold), then remove the corresponding rows from S and entries from $\mathbf{I}(x, y)$, and finally solve the reduced least-squares problem on the remaining valid measurements. This row deletion strategy preserves the least-squares structure and ensures that only valid measurements contribute to the normal estimate. However, it requires at least three non-shadowed lights at every pixel to maintain $\text{rank}(S) = 3$.

For pixels near shadow boundaries where intensities are low but non-zero, a weighted least-squares approach provides a smoother alternative:

$$\hat{\mathbf{g}} = \arg \min_{\mathbf{g}} \sum_{i=1}^m w_i (I_i - \kappa_i \mathbf{L}_i^T \mathbf{g})^2 \quad (21)$$

where weights w_i are set based on confidence (e.g., $w_i = \max(0, \mathbf{n} \cdot \mathbf{L}_i)$) or based on intensity magnitude). This provides a smooth transition between fully illuminated and shadowed regions.

2.2 Gradient Integration via Poisson Equation

We now have the normals $\mathbf{g}(x, y)$ from photometric stereo, but our ultimate goal is a height map $z(x, y)$. The normals allow us to compute gradient estimates $(p, q) = (\partial z / \partial x, \partial z / \partial y)$, but real measurements are noisy and generally not integrable—the curl $\partial_y p - \partial_x q \neq 0$. This section develops the variational framework that projects noisy gradients onto the closest integrable field.

2.2.1 Variational Formulation

Given noisy gradient estimates $(\hat{p}, \hat{q})$ from photometric stereo, we seek a height field $z(x, y)$ whose gradients best match the measurements in a least-squares sense. This leads to the variational problem:

$$\min_z \mathcal{E}[z] = \iint_{\Omega} [(\partial_x z - \hat{p})^2 + (\partial_y z - \hat{q})^2] dx dy \quad (22)$$

The functional $\mathcal{E}[z]$ measures the total squared error between the true gradients $(\partial_x z, \partial_y z)$ of the unknown surface and the measured estimates $(\hat{p}, \hat{q})$. By minimizing over all possible height functions z , we find the surface whose gradients are closest to the measurements in the L^2 norm.

The integrand $\mathcal{L} = (\partial_x z - \hat{p})^2 + (\partial_y z - \hat{q})^2$ depends on z only through its first derivatives $z_x = \partial_x z$ and $z_y = \partial_y z$. This structure is characteristic of problems in the calculus of variations, and the minimizer satisfies the Euler–Lagrange equation.

2.2.2 Euler–Lagrange Derivation

For a functional of the form $\mathcal{E}[z] = \iint \mathcal{L}(z, z_x, z_y) dx dy$, the necessary condition for a minimum is the Euler–Lagrange equation:

$$\frac{\partial \mathcal{L}}{\partial z} - \frac{\partial}{\partial x} \left(\frac{\partial \mathcal{L}}{\partial z_x} \right) - \frac{\partial}{\partial y} \left(\frac{\partial \mathcal{L}}{\partial z_y} \right) = 0 \quad (23)$$

Since our Lagrangian $\mathcal{L} = (z_x - \hat{p})^2 + (z_y - \hat{q})^2$ does not depend on z directly, we have $\partial \mathcal{L} / \partial z = 0$. Computing the remaining partial derivatives:

$$\frac{\partial \mathcal{L}}{\partial z_x} = 2(z_x - \hat{p}), \quad \frac{\partial \mathcal{L}}{\partial z_y} = 2(z_y - \hat{q}) \quad (24)$$

Substituting into the Euler–Lagrange equation:

$$\frac{\partial}{\partial x} [2(z_x - \hat{p})] + \frac{\partial}{\partial y} [2(z_y - \hat{q})] = 0 \quad (25)$$

Expanding and rearranging:

$$\partial_x^2 z + \partial_y^2 z = \partial_x \hat{p} + \partial_y \hat{q} \quad (26)$$

This is the Poisson equation:

$$\nabla^2 z = f(x, y), \quad \text{where } f = \nabla \cdot (\hat{p}, \hat{q}) = \frac{\partial \hat{p}}{\partial x} + \frac{\partial \hat{q}}{\partial y} \quad (27)$$

Here $\nabla^2 = \partial_x^2 + \partial_y^2$ is the Laplacian operator and $f(x, y)$ is the divergence of the measured gradient field. The Poisson equation (27) arises because an integrable vector field must be curl-free ($\partial_y \hat{p} - \partial_x \hat{q} = 0$). When the measured gradients violate this condition due to noise, solving the Poisson problem projects $(\hat{p}, \hat{q})$ onto the closest integrable field in the L^2 sense.

2.3 Boundary Condition Theory

The Poisson equation $\nabla^2 z = f$ admits a unique solution only when supplemented with boundary conditions on $\partial\Omega$. We consider three types that arise in surface reconstruction: Dirichlet, Neumann, and periodic conditions. The choice of boundary conditions depends on the physical assumptions about the surface at the image boundary and impacts both solution uniqueness and numerical implementation.

2.3.1 Dirichlet Boundary Conditions

Dirichlet conditions prescribe the solution value on the boundary:

$$z(x, y) = g(x, y), \quad (x, y) \in \partial\Omega \quad (28)$$

where g is a given function. In the homogeneous case $g \equiv 0$, the surface is pinned to zero height along the entire boundary. This is appropriate when the reconstructed object is known to be flat at the image edges or when the object of interest is contained entirely within the interior of the domain.

For photometric stereo, homogeneous Dirichlet conditions assume that the surface height is zero (or a known constant) at the image boundary. This is suitable for objects that sit on a flat background plane or when the region of interest is masked to exclude edge artifacts. Dirichlet problems always have a unique solution given smooth data.

2.3.2 Neumann Boundary Conditions

Neumann conditions prescribe the normal derivative at the boundary:

$$\left. \frac{\partial z}{\partial \mathbf{n}} \right|_{\partial\Omega} = h(x, y), \quad (x, y) \in \partial\Omega \quad (29)$$

where $\mathbf{n}$ is the outward unit normal to $\partial\Omega$ and h specifies the boundary flux. The homogeneous case $h \equiv 0$ imposes zero normal derivative, meaning the surface approaches the boundary with zero slope in the perpendicular direction.

Zero-flux Neumann conditions arise when we have no information about the surface beyond the image boundary. The natural continuation of the gradient field is to assume it stays flat, preventing artificial discontinuities. However, Neumann conditions leave the solution determined only up to an additive constant—we can recover the surface shape but not its absolute height. This lack of absolute height information is resolved by simply setting the average of the entire reconstructed depth profile to zero.

2.3.3 Periodic Boundary Conditions

Periodic conditions impose that the solution and its derivatives wrap around at opposite boundaries:

$$z(0, y) = z(L_x, y), \quad z(x, 0) = z(x, L_y), \quad \text{and similarly for derivatives} \quad (30)$$

This condition arises naturally when using Fourier-based (FFT) solvers, which assume the domain tiles infinitely in all directions. Under periodicity, the Discrete Fourier Transform diagonalizes the Laplacian operator, enabling $\mathcal{O}(N \log N)$ solution complexity.

Periodic conditions are rarely physical for real surfaces but are computationally convenient. They work well when the surface gradients decay to zero near the boundary (e.g., smooth objects

centered in the image) but can introduce artifacts when the surface value or gradient differs significantly between opposite edges. In practice, we often accept these artifacts in exchange for FFT solver speed, especially when the object of interest is well-separated from the image boundary.

2.3.4 Compatibility and Uniqueness

The three boundary condition types have different uniqueness and compatibility properties, which determine when a solution exists and whether it is unique up to an additive constant.

Dirichlet conditions guarantee a unique solution for any smooth right-hand side f and boundary data g . No additional constraints are required—the boundary values fully determine the solution.

Neumann conditions require a compatibility constraint. Applying the divergence theorem to the Poisson equation:

$$\iint_{\Omega} \nabla^2 z \, dA = \oint_{\partial\Omega} \frac{\partial z}{\partial n} \, ds \quad (31)$$

Substituting $\nabla^2 z = f$ and the Neumann condition $\partial z / \partial n = h$:

$$\iint_{\Omega} f \, dA = \oint_{\partial\Omega} h \, ds \quad (32)$$

For homogeneous Neumann conditions ($h = 0$), solvability requires:

$$\iint_{\Omega} f \, dA = 0 \quad (33)$$

In other words, the source term must integrate to zero over the domain. Even when this condition is satisfied, the solution is unique only up to an additive constant—any $z + c$ is also a solution. We select the unique solution with zero mean.

Periodic conditions have the same compatibility requirement as homogeneous Neumann. Since the domain wraps around with no boundary flux, the integral constraint $\iint f \, dA = 0$ must hold. In the FFT formulation, this corresponds to the DC component $\hat{F}[0, 0] = 0$ —the zero-frequency Fourier coefficient must vanish. The FFT solver enforces this by setting $\hat{F}[0, 0] = 0$ before division by the eigenvalues. Like Neumann, periodic conditions also leave an additive constant ambiguity, resolved by zero-mean normalization.

2.4 Analytical Solution via Separation of Variables

Before developing numerical integration schemes to solve the Poisson equation for our photometric stereo application, we first derive an analytical solution as a baseline for comparison—and, candidly, as an exercise to demonstrate mathematical rigor in the realm of partial differential equations.

The method of separation of variables yields closed-form expressions for problems with homogeneous boundary conditions on rectangular domains. By expanding the solution as a sum of orthogonal eigenfunctions, we obtain exact modal coefficients that serve as ground truth for validating numerical implementations. This section provides the theoretical foundation but will not be directly referenced in subsequent chapters; nevertheless, the analytical benchmark remains essential for verifying solver correctness during development.

2.4.1 Eigenfunction Expansion

Consider the 2D Poisson equation on a rectangular domain $\Omega = [0, L_x] \times [0, L_y]$ with homogeneous Dirichlet boundary conditions:

$$\nabla^2 z = f(x, y), \quad (x, y) \in \Omega, \quad z|_{\partial\Omega} = 0 \quad (34)$$

Using separation of variables, we assume $z(x, y) = X(x)Y(y)$. The homogeneous boundary conditions require $X(0) = X(L_x) = 0$ and $Y(0) = Y(L_y) = 0$, which are satisfied by sine functions. The solution can be written as an eigenfunction expansion:

$$z(x, y) = \sum_{m=1}^{\infty} \sum_{n=1}^{\infty} A_{mn} \sin\left(\frac{m\pi x}{L_x}\right) \sin\left(\frac{n\pi y}{L_y}\right) \quad (35)$$

Figure 1 illustrates how higher modes introduce additional oscillations. In one dimension, $\sin(m\pi x/L)$ has m half-cycles across the domain, so higher m values correspond to finer spatial detail.

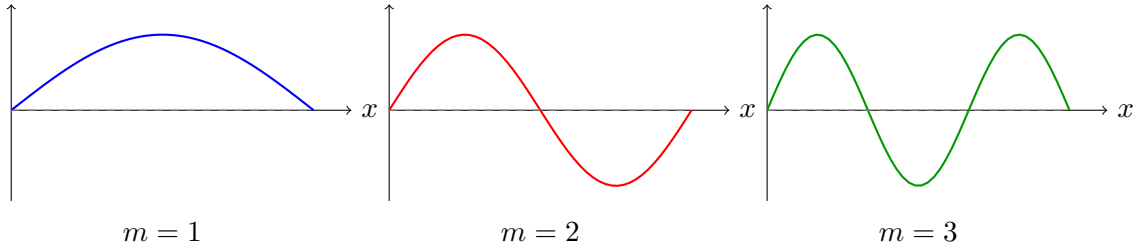


Figure 1: Eigenfunctions $\sin(m\pi x/L)$ for $m = 1, 2, 3$. The 2D eigenfunctions are products: $\sin(m\pi x/L_x) \sin(n\pi y/L_y)$. Higher modes oscillate more rapidly and contribute less to the solution because their eigenvalues $\lambda_{mn} \propto m^2 + n^2$ appear in the denominator.

The eigenfunctions form an orthonormal basis for the solution space. Each eigenfunction satisfies the Laplacian eigenvalue problem with eigenvalue $-\lambda_{mn}$ where:

$$\lambda_{mn} = \frac{m^2\pi^2}{L_x^2} + \frac{n^2\pi^2}{L_y^2} \quad (36)$$

Substituting the expansion into the Poisson equation and projecting onto each eigenfunction yields the modal amplitudes:

$$A_{mn} = -\frac{F_{mn}}{\lambda_{mn}}, \quad \text{where } F_{mn} = \frac{4}{L_x L_y} \int_0^{L_x} \int_0^{L_y} f(x, y) \sin\left(\frac{m\pi x}{L_x}\right) \sin\left(\frac{n\pi y}{L_y}\right) dy dx \quad (37)$$

The coefficient F_{mn} is the projection of the source term f onto each eigenfunction. This is precisely what FFT-based solvers compute numerically: the Discrete Fourier Transform projects the discretized f onto sinusoidal basis functions, and division by the eigenvalues λ_{mn} yields the solution coefficients.

3 Numerical Methods

Having established the mathematical framework for photometric stereo and gradient integration, we now turn to the numerical methods that enable practical computation. This chapter presents the techniques for converting the continuous Poisson equation into discrete systems that computers can solve efficiently.

We begin with the fundamental discretization approach—converting continuous derivatives to finite differences on a grid. This lays the groundwork for understanding all subsequent solvers. We then present three distinct Poisson solvers, each with different boundary condition assumptions and computational trade-offs. Finally, we address regularization techniques for handling noisy real-world data.

3.1 From Continuous PDE to Discrete Grid

The continuous Poisson equation $\nabla^2 z = f$ cannot be solved directly by computers, which operate on discrete values. The finite difference method replaces continuous derivatives with algebraic approximations computed on a regular grid of sample points. This section derives these approximations and shows how they lead to a large linear system.

3.1.1 Approximating Derivatives (Taylor Series)

Consider a function $z(x)$ sampled on a uniform grid with spacing h . To approximate derivatives, we use Taylor series expansions about a point x :

$$z(x+h) = z(x) + hz'(x) + \frac{h^2}{2}z''(x) + \frac{h^3}{6}z'''(x) + \mathcal{O}(h^4) \quad (38)$$

$$z(x-h) = z(x) - hz'(x) + \frac{h^2}{2}z''(x) - \frac{h^3}{6}z'''(x) + \mathcal{O}(h^4) \quad (39)$$

Adding these two equations eliminates the odd-order terms:

$$z(x+h) + z(x-h) = 2z(x) + h^2z''(x) + \mathcal{O}(h^4) \quad (40)$$

Solving for the second derivative yields the central difference formula:

$$z''(x) = \frac{z(x+h) - 2z(x) + z(x-h)}{h^2} + \mathcal{O}(h^2) \quad (41)$$

This approximation is second-order accurate—the error decreases quadratically as the grid spacing h is refined. On a discrete grid where $z_i = z(x_i)$, we write this as:

$$z''_i \approx \frac{z_{i+1} - 2z_i + z_{i-1}}{h^2} \quad (42)$$

3.1.2 The 5-Point Stencil Pattern

In two dimensions, the Laplacian combines second derivatives in both coordinate directions, capturing how the function curves in the x - and y -directions:

$$\nabla^2 z = \frac{\partial^2 z}{\partial x^2} + \frac{\partial^2 z}{\partial y^2} \quad (43)$$

Applying the central difference formula (41) in both directions at grid point (i, j) :

$$\nabla^2 z \Big|_{i,j} \approx \frac{z_{i+1,j} - 2z_{i,j} + z_{i-1,j}}{(\Delta x)^2} + \frac{z_{i,j+1} - 2z_{i,j} + z_{i,j-1}}{(\Delta y)^2} \quad (44)$$

For uniform spacing $h = \Delta x = \Delta y$, this simplifies to the 5-point stencil:

$$\boxed{\nabla^2 z \Big|_{i,j} \approx \frac{z_{i+1,j} + z_{i-1,j} + z_{i,j+1} + z_{i,j-1} - 4z_{i,j}}{h^2}} \quad (45)$$

The stencil pattern represents a local finite difference operator that combines values from the center point and its four nearest neighbors on the grid. Figure 2 visualizes the coefficient structure:

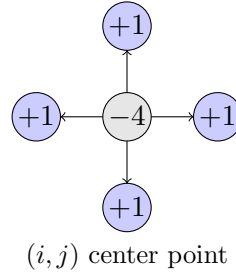


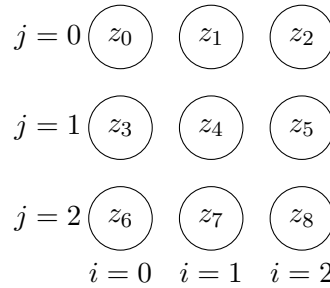
Figure 2: The 5-point Laplacian stencil weights.

The stencil computes a weighted average of the four neighbors minus four times the center value, all divided by h^2 . This approximates how the curvature (Laplacian) at a point relates to its deviation from the average of its neighbors.

3.1.3 Building the Linear System

To solve the Poisson equation numerically, we must convert the stencil operations into a single large linear system $Az = \mathbf{f}$. This requires mapping the 2D grid to a 1D vector and translating neighbor relationships into matrix entries.

Consider a simple 3×3 grid with 9 unknowns. Using row-major ordering, we assign linear index $k = i + j \cdot W$ where $W = 3$ is the grid width:



For an interior point like z_4 (at position $i = 1, j = 1$), the 5-point stencil couples it to its four neighbors:

$$\frac{z_5 + z_3 + z_7 + z_1 - 4z_4}{h^2} = f_4 \quad (46)$$

This becomes one row of the linear system. Rearranging with unknowns on the left:

$$\frac{1}{h^2}z_1 + \frac{1}{h^2}z_3 - \frac{4}{h^2}z_4 + \frac{1}{h^2}z_5 + \frac{1}{h^2}z_7 = f_4 \quad (47)$$

Writing this for all 9 grid points simultaneously yields the matrix equation $\mathbf{A}\mathbf{z} = \mathbf{f}$. The matrix $\mathbf{A}$ has a characteristic block structure:

$$\mathbf{A} = \frac{1}{h^2} \begin{bmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{bmatrix} \quad (48)$$

Each row corresponds to one grid point. The diagonal entry is -4 (center weight), entries at ± 1 positions connect horizontal neighbors (same row), and entries at $\pm W$ positions connect vertical neighbors (adjacent rows). The matrix is sparse—only 5 entries per row instead of N —which is crucial for efficiency on large grids.

Boundary conditions modify the right-hand side $\mathbf{f}$ or the matrix structure itself. For Dirichlet conditions, known boundary values move to the right-hand side. For Neumann conditions, stencil weights at boundaries are adjusted to enforce derivative constraints. The specific modifications are detailed in the solver sections that follow.

3.2 Solver 1: FFT-Based (Spectral Method)

The FFT-based solver exploits the fact that the Laplacian operator is diagonalized in the Fourier domain. This transforms the Poisson PDE into simple algebraic division, enabling $\mathcal{O}(N \log N)$ solution complexity. This is our fastest solver and the default choice when periodic boundary conditions are acceptable.

3.2.1 Mathematical Approach

The key insight is that Fourier basis functions (complex exponentials) are eigenfunctions of the Laplacian. Applying the Fourier transform to both sides of $\nabla^2 z = f$:

$$\mathcal{F}\{\nabla^2 z\} = \mathcal{F}\{f\} \quad (49)$$

The Laplacian in the frequency domain becomes multiplication by eigenvalues. Let $\hat{Z}(\mathbf{k})$ and $\hat{F}(\mathbf{k})$ denote the Fourier transforms of z and f at frequency $\mathbf{k} = (k_x, k_y)$:

$$-|\mathbf{k}|^2 \hat{Z}(\mathbf{k}) = \hat{F}(\mathbf{k}) \quad (50)$$

where $|\mathbf{k}|^2 = k_x^2 + k_y^2$ is the squared frequency magnitude. Solving for the solution in frequency space:

$$\hat{Z}(\mathbf{k}) = \frac{\hat{F}(\mathbf{k})}{-|\mathbf{k}|^2} \quad (51)$$

The solution is obtained by inverse Fourier transform:

$$z(x, y) = \mathcal{F}^{-1} \left\{ \frac{\hat{F}(\mathbf{k})}{-|\mathbf{k}|^2} \right\} \quad (52)$$

This converts the PDE (which requires solving a large linear system) into three simple operations: forward FFT, element-wise division, and inverse FFT.

3.2.2 Boundary Condition: Periodic

The FFT inherently assumes that the signal is periodic in all directions. This means the solver implicitly enforces:

$$z(0, y) = z(L_x, y), \quad z(x, 0) = z(x, L_y), \quad \text{and similarly for all derivatives} \quad (53)$$

Periodic boundary conditions are rarely physical for real surfaces—few objects wrap around seamlessly at their edges. However, they work well when the object of interest is centered in the image with gradients decaying to near-zero at the boundaries.

A critical issue arises at the DC component ($\mathbf{k} = (0, 0)$): the eigenvalue $|\mathbf{k}|^2 = 0$, creating a division by zero. This corresponds to the compatibility condition from Section 2.3—periodic problems determine the solution only up to an additive constant. We handle this by setting $\hat{F}[0, 0] = 0$ to enforce zero-mean source (compatibility), setting the eigenvalue $\lambda[0, 0] = 1$ to avoid division by zero, and the resulting $\hat{Z}[0, 0] = 0$ gives a zero-mean solution.

3.2.3 Algorithm Steps

The complete FFT solver procedure is shown in Algorithm 1. The algorithm requires only two FFT operations (steps 2 and 6), each with $\mathcal{O}(N \log N)$ complexity, making this solver extremely fast for typical image sizes.

Algorithm 1 FFT-Based Poisson Solver

Require: Gradient estimates (p, q) from photometric stereo

Ensure: Height field z

- 1: Compute divergence: $f \leftarrow \partial_x p + \partial_y q$
 - 2: $\hat{F} \leftarrow \text{FFT2D}(f)$
 - 3: Construct eigenvalues: $\lambda_{k_x, k_y} \leftarrow -4\pi^2(k_x^2/L_x^2 + k_y^2/L_y^2)$
 - 4: Handle DC component: $\lambda[0, 0] \leftarrow 1$; $\hat{F}[0, 0] \leftarrow 0$
 - 5: $\hat{Z} \leftarrow \hat{F}/\lambda$
 - 6: $z \leftarrow \text{IFFT2D}(\hat{Z})$
 - 7: $z \leftarrow \text{Re}(z)$
 - 8: **return** z
-

3.3 Solver 2: Finite Difference (Dirichlet Boundaries)

The finite difference solver with Dirichlet boundary conditions constructs and solves the sparse linear system $A\mathbf{z} = \mathbf{f}$ derived in Section 3.1. Unlike the FFT approach, this method uses iterative linear algebra solvers that exploit the matrix sparsity, making it less efficient but more flexible in handling boundary conditions.

3.3.1 Mathematical Approach

We solve the discrete Poisson equation using the 5-point stencil developed in Section 3.1. For interior grid points, the stencil equation is:

$$\frac{z_{i+1,j} + z_{i-1,j} + z_{i,j+1} + z_{i,j-1} - 4z_{i,j}}{h^2} = f_{i,j} \quad (54)$$

This produces a sparse $N \times N$ linear system where $N = H \cdot W$ is the total number of grid points. The matrix A is symmetric negative definite, guaranteeing convergence of iterative methods.

We solve this system using the Conjugate Gradient (CG) method, an iterative algorithm that finds the solution by successively minimizing the quadratic form $\frac{1}{2}\mathbf{z}^T A\mathbf{z} - \mathbf{f}^T \mathbf{z}$. CG generates a sequence of search directions that are mutually conjugate (orthogonal with respect to A), ensuring that each iteration makes progress in a new direction. For symmetric positive definite systems, CG converges in at most N iterations, though in practice far fewer iterations are needed when the matrix is well-conditioned.

Rather than explicitly forming the matrix A (which would require storing $5N$ entries), we define a matrix-free linear operator that applies the 5-point stencil to any input vector. This reduces memory usage from $\mathcal{O}(N)$ to $\mathcal{O}(1)$ for the operator itself and enables efficient matrix-vector products.

3.3.2 Boundary Condition: Fixed Values

Dirichlet conditions prescribe the solution value on the boundary:

$$z(x, y) = g(x, y), \quad (x, y) \in \partial\Omega \quad (55)$$

In the homogeneous case ($g \equiv 0$), the surface is pinned to zero height along the entire boundary. For photometric stereo, this is appropriate when the reconstructed object sits on a flat background or when the region of interest is masked to exclude edge artifacts.

Implementation is straightforward: boundary points are held fixed at the prescribed value g , and only interior points are solved. The stencil at interior points adjacent to boundaries incorporates the known boundary values into the right-hand side. Figure 3 illustrates this for a point near the boundary.

For a point next to the left boundary ($i = 1$), the stencil becomes:

$$\frac{z_{2,j} + g_{0,j} + z_{1,j+1} + z_{1,j-1} - 4z_{1,j}}{h^2} = f_{1,j} \quad (56)$$

The known boundary value $g_{0,j}$ moves to the right-hand side: $f'_{1,j} = f_{1,j} - g_{0,j}/h^2$. This effectively reduces the system size to interior points only.

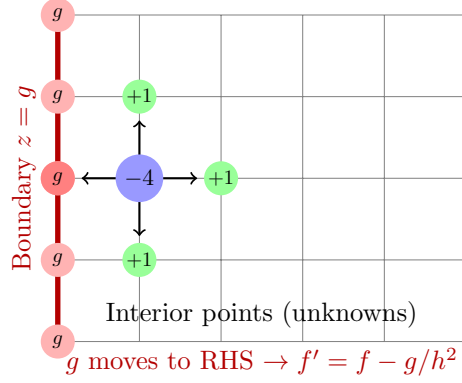


Figure 3: Dirichlet boundary conditions: The known boundary value g (red) is incorporated into the right-hand side, and only interior points (blue) are solved as unknowns.

3.3.3 Algorithm Steps

The complete finite difference Dirichlet solver is shown in Algorithm 2. The solver uses Conjugate Gradient (CG) for symmetric positive definite systems or GMRES for general sparse systems. The computational cost is $\mathcal{O}(N \cdot k)$ where k is the number of CG iterations needed for convergence. Typically $k \ll N$, but the method is slower than FFT for large grids. The advantage is the ability to enforce exact boundary values.

Algorithm 2 Finite Difference Poisson Solver (Dirichlet BC)

Require: Gradient estimates (p, q) , boundary values g

Ensure: Height field z with $z|_{\partial\Omega} = g$

- 1: Compute divergence: $f \leftarrow \partial_x p + \partial_y q$
 - 2: Define matrix-free operator $A : z \mapsto \nabla_h^2 z$ (5-point stencil)
 - 3: Modify RHS for boundary: $\mathbf{f}' \leftarrow \mathbf{f} - A_{\text{boundary}} \mathbf{g}$
 - 4: Initialize: $z_0 \leftarrow 0$ for interior points
 - 5: Solve: $\mathbf{z}_{\text{int}} \leftarrow \text{CG}(A_{\text{int}}, \mathbf{f}', z_0, \text{tol})$
 - 6: Combine: $z \leftarrow z_{\text{int}} \cup g$
 - 7: **return** z
-

3.4 Solver 3: DCT-Based (Neumann Boundaries)

The Neumann solver handles problems where boundary derivatives, rather than values, are specified. This is often more physically realistic for photometric stereo, where we typically have no information about the absolute height at image boundaries—only the surface gradients.

Unlike the FFT solver which uses iterative methods or complex boundary handling, we use the Discrete Cosine Transform (DCT), which naturally enforces Neumann conditions through its basis functions. The key insight is that cosine basis functions have zero derivative at both endpoints:

$$\left. \frac{d}{dx} \cos\left(\frac{n\pi x}{L}\right) \right|_{x=0} = \left. \frac{d}{dx} \cos\left(\frac{n\pi x}{L}\right) \right|_{x=L} = 0 \quad (57)$$

This means the DCT *implicitly* enforces homogeneous Neumann boundary conditions $\partial z / \partial n = 0$ without any iterative solver or explicit boundary handling. In contrast, the FFT uses complex exponentials which implicitly enforce *periodic* boundary conditions.

3.4.1 Mathematical Formulation

Like the FFT solver, the DCT approach converts the Poisson equation into algebraic operations in the spectral domain. The DCT achieves $\mathcal{O}(N \log N)$ complexity while naturally enforcing zero-flux (Neumann) conditions at the domain boundaries.

The Type-II DCT (forward) and Type-III DCT (inverse, also called IDCT) of a 2D field $f_{i,j}$ are defined as:

$$\hat{F}_{k,l} = \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} f_{i,j} \cos\left[\frac{\pi k(i + \frac{1}{2})}{N_x}\right] \cos\left[\frac{\pi l(j + \frac{1}{2})}{N_y}\right] \quad (58)$$

$$f_{i,j} = \sum_{k=0}^{N_x-1} \sum_{l=0}^{N_y-1} w_k w_l \hat{F}_{k,l} \cos\left[\frac{\pi k(i + \frac{1}{2})}{N_x}\right] \cos\left[\frac{\pi l(j + \frac{1}{2})}{N_y}\right] \quad (59)$$

where $w_0 = 1/N$ and $w_k = 2/N$ for $k > 0$ are normalization weights.

The discrete Laplacian operator acting on the cosine basis yields eigenvalues:

$$\lambda_{k,l} = -\frac{2}{\Delta x^2} \left(1 - \cos \frac{\pi k}{N_x}\right) - \frac{2}{\Delta y^2} \left(1 - \cos \frac{\pi l}{N_y}\right) \quad (60)$$

These eigenvalues arise from the second-order central difference stencil combined with the Neumann boundary condition (ghost point reflection). At $k = l = 0$, we have $\lambda_{0,0} = 0$, corresponding to the null-space constant inherent to Neumann problems.

The spectral solution follows directly:

$$\hat{Z}_{k,l} = \frac{\hat{F}_{k,l}}{\lambda_{k,l}}, \quad (k,l) \neq (0,0) \quad (61)$$

The DC component $\hat{Z}_{0,0}$ is set to zero (or any constant), which corresponds to choosing the particular solution with zero mean.

3.4.2 Boundary Condition: Zero Flux

Neumann conditions specify that the normal derivative vanishes at the boundary:

$$\left. \frac{\partial z}{\partial n} \right|_{\partial\Omega} = 0 \quad (62)$$

Physically, this means the surface approaches the boundary with zero slope perpendicular to the edge. For photometric stereo, this is appropriate when we have no information about absolute heights at the image boundary—we only know that the surface should extend smoothly beyond the observed region.

Unlike periodic conditions (which cause wrap-around artifacts) or Dirichlet conditions (which force the boundary to a prescribed value), Neumann conditions impose no artificial constraints on boundary heights. The surface is free to take any shape at the edges, subject only to a zero-slope constraint.

A crucial requirement is the *compatibility condition*:

$$\iint_{\Omega} f \, dA = 0 \quad (63)$$

This follows from the divergence theorem applied to $\nabla^2 z = f$ with zero-flux boundaries. If the input f does not satisfy this condition, we enforce it by subtracting the mean: $f \leftarrow f - \text{mean}(f)$. After solving, the result is determined only up to an additive constant, which we fix by mean-centering.

3.4.3 Algorithm

The complete DCT-based solver procedure is shown in Algorithm 3. The algorithm requires only two DCT operations (forward and inverse), each with $\mathcal{O}(N \log N)$ complexity, making this solver as fast as the FFT approach. The key steps are: (1) enforce compatibility by mean-centering the divergence field, (2) transform to frequency domain via DCT-II, (3) divide by eigenvalues (spectral solve), (4) transform back via inverse DCT-II, and (5) mean-center the result to fix the additive constant ambiguity.

Unlike an iterative finite difference approach which requires tuning convergence tolerances and maximum iteration counts, the DCT method is *direct*—it always converges in a single pass with no parameter tuning required. This makes it both faster and more robust than iterative Neumann solvers.

Algorithm 3 DCT-Based Poisson Solver (Neumann BC)

Require: Gradient estimates (p, q) from photometric stereo

Ensure: Height field z with $\partial z / \partial n|_{\partial\Omega} = 0$

- 1: Compute divergence: $f \leftarrow \partial_x p + \partial_y q$
 - 2: Enforce compatibility: $f \leftarrow f - \text{mean}(f)$
 - 3: Compute eigenvalues: $\lambda_{ij} \leftarrow -\frac{2}{\Delta x^2}(1 - \cos \frac{\pi i}{N_x}) - \frac{2}{\Delta y^2}(1 - \cos \frac{\pi j}{N_y})$
 - 4: Forward transform: $\hat{F} \leftarrow \text{DCT-II}(f)$
 - 5: Spectral solve: $\hat{Z}_{ij} \leftarrow \hat{F}_{ij} / \lambda_{ij}$, with $\hat{Z}_{00} \leftarrow 0$
 - 6: Inverse transform: $z \leftarrow \text{IDCT-II}(\hat{Z})$
 - 7: Center solution: $z \leftarrow z - \text{mean}(z)$
 - 8: **return** z
-

3.5 Solver 4: CG-Iterative (Dirichlet Boundaries)

The Conjugate Gradient (CG) iterative solver provides an alternative approach to the Poisson equation, using iterative refinement rather than direct factorization or spectral transforms. This method achieves higher precision than direct sparse solvers while maintaining the flexibility of arbitrary boundary conditions.

3.5.1 Mathematical Formulation

The CG method solves the sparse linear system arising from finite difference discretization of the Laplacian. For a grid of size $N_x \times N_y$, we discretize the Poisson equation $\nabla^2 z = f$ using the standard 5-point stencil:

$$\frac{z_{i-1,j} - 2z_{i,j} + z_{i+1,j}}{\Delta x^2} + \frac{z_{i,j-1} - 2z_{i,j} + z_{i,j+1}}{\Delta y^2} = f_{i,j} \quad (64)$$

This yields a sparse linear system $\mathbf{A}\mathbf{z} = \mathbf{b}$ where $\mathbf{A}$ is an $N \times N$ matrix (with $N = N_x \cdot N_y$) having at most 5 non-zero entries per row. The matrix $\mathbf{A}$ is symmetric positive semi-definite, making it suitable for the Conjugate Gradient method.

3.5.2 Dirichlet Boundary Conditions

We enforce Dirichlet conditions by setting boundary values to zero:

$$z(x, y) = 0 \quad \text{for } (x, y) \in \partial\Omega \quad (65)$$

In matrix form, boundary rows of $\mathbf{A}$ are replaced with identity rows (diagonal = 1, off-diagonal = 0), and corresponding entries in $\mathbf{b}$ are set to zero. This ensures boundary unknowns remain fixed throughout the iteration.

3.5.3 Conjugate Gradient Iteration

For symmetric positive definite systems, CG converges in at most N iterations, though in practice far fewer are needed. The algorithm minimizes the quadratic form:

$$\phi(\mathbf{z}) = \frac{1}{2} \mathbf{z}^T \mathbf{A} \mathbf{z} - \mathbf{b}^T \mathbf{z} \quad (66)$$

Each iteration updates the solution along conjugate directions, guaranteeing monotonic decrease in the error norm $\|\mathbf{z}^{(k)} - \mathbf{z}^*\|_{\mathbf{A}}$.

The convergence rate depends on the condition number $\kappa(\mathbf{A})$. For the discrete Laplacian, $\kappa \sim \mathcal{O}(N)$, giving convergence in $\mathcal{O}(\sqrt{N})$ iterations for a tolerance of 10^{-10} . In practice, we observe 200–400 iterations for 128×128 grids.

3.5.4 Algorithm

Algorithm 4 CG-Iterative Poisson Solver (Dirichlet BC)

Require: Gradients (p, q) , grid spacing $(\Delta x, \Delta y)$, tolerance ϵ **Ensure:** Height field z with $z|_{\partial\Omega} = 0$

- 1: Compute divergence: $\mathbf{b} \leftarrow \partial_x p + \partial_y q$
 - 2: Build sparse Laplacian matrix $\mathbf{A}$ with 5-point stencil
 - 3: Set boundary rows to identity, boundary RHS to zero
 - 4: Initialize: $\mathbf{z}^{(0)} \leftarrow \mathbf{0}$, $\mathbf{r}^{(0)} \leftarrow \mathbf{b}$, $\mathbf{p}^{(0)} \leftarrow \mathbf{r}^{(0)}$
 - 5: **while** $\|\mathbf{r}^{(k)}\| > \epsilon\|\mathbf{b}\|$ **do**
 - 6: $\alpha_k \leftarrow \frac{\mathbf{r}^{(k)T}\mathbf{r}^{(k)}}{\mathbf{p}^{(k)T}\mathbf{A}\mathbf{p}^{(k)}}$
 - 7: $\mathbf{z}^{(k+1)} \leftarrow \mathbf{z}^{(k)} + \alpha_k\mathbf{p}^{(k)}$
 - 8: $\mathbf{r}^{(k+1)} \leftarrow \mathbf{r}^{(k)} - \alpha_k\mathbf{A}\mathbf{p}^{(k)}$
 - 9: $\beta_k \leftarrow \frac{\mathbf{r}^{(k+1)T}\mathbf{r}^{(k+1)}}{\mathbf{r}^{(k)T}\mathbf{r}^{(k)}}$
 - 10: $\mathbf{p}^{(k+1)} \leftarrow \mathbf{r}^{(k+1)} + \beta_k\mathbf{p}^{(k)}$
 - 11: **end while**
 - 12: Center solution: $z \leftarrow z - \text{mean}(z)$
 - 13: **return** z
-

The CG solver achieves higher precision than direct methods because it iterates until the residual falls below a specified tolerance (10^{-10} in our implementation), whereas direct solvers may accumulate numerical errors through factorization. This explains the superior RMSE results observed in Section 7.

3.6 Regularization (Noise Handling)

Real photometric stereo data contains noise from sensor limitations, lighting imperfections, and violations of the Lambertian assumption. This noise propagates through the gradient estimation and into the Poisson problem, where it can cause severe artifacts in the reconstructed surface. Regularization techniques address this by penalizing solutions that fit noise too closely.

3.6.1 Why Regularization is Needed

The Poisson equation solvers developed above find the exact solution to $\nabla^2 z = f$. However, when f is contaminated with noise, the “exact” solution faithfully reproduces that noise—often with amplification. Consider the frequency-domain solution:

$$\hat{Z}(\mathbf{k}) = \frac{\hat{F}(\mathbf{k})}{-|\mathbf{k}|^2} \quad (67)$$

Low-frequency noise components (small $|\mathbf{k}|$) are divided by small eigenvalues, dramatically amplifying their contribution to the solution. A small amount of low-frequency noise in the gradient field can produce large-scale undulations in the reconstructed surface.

This is a manifestation of the Poisson equation’s ill-conditioning for noisy data. The problem is well-posed mathematically but numerically unstable: small perturbations in f cause large perturbations in z . Regularization stabilizes the problem by trading exact data fitting for solution smoothness.

3.6.2 Tikhonov Derivation

Tikhonov regularization adds a penalty term to the objective function, preferring solutions with small norm:

$$z_\lambda = \arg \min_z \{ \|\nabla^2 z - f\|^2 + \lambda \|z\|^2 \} \quad (68)$$

The first term ensures the solution approximately satisfies the Poisson equation; the second term penalizes large height values. The parameter $\lambda > 0$ controls the trade-off: larger λ produces smoother but less accurate solutions.

To find the minimizer, we take the first variation with respect to z and set it to zero, applying the calculus of variations to the biharmonic operator that arises from composing the Laplacian with itself:

$$(\nabla^4 + \lambda I)z = \nabla^2 f \quad (69)$$

In the frequency domain, this becomes:

$$\hat{Z}_\lambda(\mathbf{k}) = \frac{-|\mathbf{k}|^2 \hat{F}(\mathbf{k})}{|\mathbf{k}|^4 + \lambda} \quad (70)$$

The regularization modifies the eigenvalue division: instead of dividing by $|\mathbf{k}|^2$, we effectively divide by $|\mathbf{k}|^2 + \lambda/|\mathbf{k}|^2$. This prevents blow-up at low frequencies while preserving high-frequency detail where eigenvalues are large. The regularized solution smoothly interpolates between no modification (high frequencies) and strong damping (low frequencies).

3.6.3 Choosing the Parameter

The regularization parameter λ must balance bias against variance. Too small, and noise dominates; too large, and genuine surface features are smoothed away. We can quantify this trade-off by examining the regularized solution's behavior as a function of λ :

$$\hat{Z}_\lambda(\mathbf{k}) = \underbrace{\frac{|\mathbf{k}|^4}{|\mathbf{k}|^4 + \lambda}}_{\text{filter factor}} \cdot \hat{Z}_{\text{true}}(\mathbf{k}) + \text{noise contribution} \quad (71)$$

The filter factor approaches 1 for $|\mathbf{k}|^4 \gg \lambda$ (high frequencies, no filtering) and approaches 0 for $|\mathbf{k}|^4 \ll \lambda$ (low frequencies, strong damping). Several methods exist for selecting λ .

3.6.4 L-Curve Method

The L-curve method provides a graphical approach to parameter selection that does not require prior knowledge of the noise level. For each candidate λ , we compute both the residual norm (how well the solution fits the data) and the solution norm (how smooth the solution is), plotting them parametrically:

$$\text{L-curve : } (\log \|\nabla^2 z_\lambda - f\|, \log \|z_\lambda\|) \quad \text{for } \lambda \in [\lambda_{\min}, \lambda_{\max}] \quad (72)$$

The resulting curve typically has an L-shape: for small λ (under-regularization), the residual is small but the solution norm is large (noisy); for large λ (over-regularization), the solution is smooth but the residual is large. The optimal λ lies at the “corner” of this L-shaped curve, where the curvature:

$$\kappa(\lambda) = \frac{\rho'(\lambda)\eta''(\lambda) - \rho''(\lambda)\eta'(\lambda)}{(\rho'^2 + \eta'^2)^{3/2}} \quad (73)$$

is maximized, where $\rho(\lambda) = \log \|\nabla^2 z_\lambda - f\|$ and $\eta(\lambda) = \log \|z_\lambda\|$. This corner represents the best trade-off between fitting the data and obtaining a smooth solution.

3.6.5 Discrepancy Principle

When the noise statistics are known or can be estimated, the discrepancy principle provides a principled approach to parameter selection. The key idea is that we should not fit the data more accurately than the noise allows—doing so means we are fitting noise rather than signal.

If the noise standard deviation is σ and there are N data points, the expected squared norm of the noise is $N\sigma^2$. We choose λ such that the residual matches this expected noise level:

$$\|\nabla^2 z_\lambda - f\|^2 \approx N\sigma^2 \quad (74)$$

In practice, we solve for λ by monotonically decreasing λ from a large value until the residual first drops below the threshold $\tau = \sigma\sqrt{N}$. A safety factor is sometimes included:

$$\|\nabla^2 z_\lambda - f\| \leq \tau \cdot \delta, \quad \delta \in [1, 2] \quad (75)$$

The discrepancy principle is particularly useful when the noise level can be estimated from the data itself, such as from flat regions of the image or from repeated measurements.

3.6.6 Cross-Validation

Cross-validation uses data splitting to estimate how well a particular λ will generalize to unseen data. This approach is especially valuable when the noise characteristics are unknown or non-Gaussian.

Partition the data into K folds (typically $K = 5$ or $K = 10$). For each fold k , hold out that portion of the data, fit the regularized solution $z_\lambda^{(-k)}$ using the remaining data, and compute the prediction error on the held-out set:

$$\text{CV}(\lambda) = \frac{1}{K} \sum_{k=1}^K \|z_\lambda^{(-k)} - z_{\text{test}}^{(k)}\|^2 \quad (76)$$

Select the regularization parameter that minimizes the cross-validation error:

$$\lambda^* = \arg \min_{\lambda} \text{CV}(\lambda) \quad (77)$$

Leave-one-out cross-validation (LOOCV, $K = N$) provides an unbiased estimate but is computationally expensive. Generalized cross-validation (GCV) provides an efficient approximation for linear problems:

$$\text{GCV}(\lambda) = \frac{\|\nabla^2 z_\lambda - f\|^2 / N}{(1 - \text{tr}(A_\lambda) / N)^2} \quad (78)$$

where A_λ is the influence matrix mapping data to predictions.

In practice for photometric stereo, a reasonable starting point is $\lambda \approx 10^{-4}$ to 10^{-2} , adjusted visually based on the smoothness of the reconstructed surface.

4 Implementation

This chapter presents the algorithmic implementation of the photometric stereo pipeline. We focus on the computational steps required to go from raw images to a reconstructed 3D surface, emphasizing the data flow and key algorithmic decisions at each stage. The implementation integrates the mathematical foundations from Chapter 2 with the numerical solvers from Chapter 3.

4.1 Photometric Stereo Pipeline

The complete pipeline consists of four stages: image acquisition and preprocessing, light matrix construction, per-pixel normal estimation, and gradient field computation. Algorithm 5 provides an overview.

Algorithm 5 Photometric Stereo Pipeline

Require: Images $\{I_1, \dots, I_K\}$ under known lighting $\{\mathbf{l}_1, \dots, \mathbf{l}_K\}$

Ensure: Reconstructed height field $z(x, y)$

- 1: Preprocess images: normalization, background subtraction
 - 2: Construct light matrix $L \in \mathbb{R}^{K \times 3}$
 - 3: **for** each pixel (x, y) **do**
 - 4: Solve $L\mathbf{n} = \mathbf{i}$ for surface normal $\mathbf{n}(x, y)$
 - 5: **end for**
 - 6: Compute gradients: $p = -n_x/n_z, q = -n_y/n_z$
 - 7: Compute divergence: $f = \partial_x p + \partial_y q$
 - 8: Solve Poisson equation: $\nabla^2 z = f$
 - 9: **return** z
-

4.1.1 Image Acquisition and Preprocessing

The input consists of K grayscale images of a static scene, each captured under a different known lighting direction. For reliable normal estimation, we require $K \geq 3$ non-coplanar light sources, though $K = 4$ to 8 provides better noise robustness.

Several preprocessing steps are essential for high-quality reconstruction. First, intensity normalization converts raw pixel values to the range $[0, 1]$ and applies gamma correction if the camera response is nonlinear. Second, background subtraction removes ambient illumination by subtracting an image captured with all lights off, or by estimating a low-frequency background model. Third, shadow detection identifies pixels where the surface is in shadow (intensity below a threshold); these measurements violate the Lambertian model and should be excluded from the least-squares fit. Finally, saturation handling excludes pixels at maximum intensity, as they provide no gradient information.

For each pixel location (x, y) , we collect the intensity vector:

$$\mathbf{i}(x, y) = [I_1(x, y), I_2(x, y), \dots, I_K(x, y)]^T \quad (79)$$

4.1.2 Light Matrix Construction

The lighting directions must be known or calibrated. Each light direction $\mathbf{l}_k = (\ell_{k,x}, \ell_{k,y}, \ell_{k,z})^T$ is a unit vector pointing from the surface toward the light source. These are assembled into the

light matrix:

$$L = \begin{bmatrix} \ell_{1,x} & \ell_{1,y} & \ell_{1,z} \\ \ell_{2,x} & \ell_{2,y} & \ell_{2,z} \\ \vdots & \vdots & \vdots \\ \ell_{K,x} & \ell_{K,y} & \ell_{K,z} \end{bmatrix} \in \mathbb{R}^{K \times 3} \quad (80)$$

For well-conditioned normal estimation, the light directions should span 3D space. The condition number $\kappa(L^T L)$ indicates sensitivity to noise—lower is better. Optimal configurations distribute lights uniformly over the hemisphere, avoiding coplanar arrangements.

Common calibration methods include using a reference sphere with known geometry (the “chrome ball” technique) or directly measuring light positions with a goniometer.

4.1.3 Per-Pixel Normal Estimation

At each pixel, we solve the overdetermined system $L\mathbf{n} = \mathbf{i}$ for the surface normal $\mathbf{n}$. The Lambertian model gives:

$$I_k(x, y) = \rho(x, y) \cdot \mathbf{l}_k \cdot \mathbf{n}(x, y) \quad (81)$$

where ρ is the albedo (reflectivity) at that pixel. We solve for the product $\mathbf{m} = \rho\mathbf{n}$ using least squares:

$$\mathbf{m}^* = (L^T L)^{-1} L^T \mathbf{i} = L^\dagger \mathbf{i} \quad (82)$$

where $L^\dagger$ is the Moore-Penrose pseudoinverse. The albedo and normal are then separated:

$$\rho = \|\mathbf{m}^*\|, \quad \mathbf{n} = \frac{\mathbf{m}^*}{\|\mathbf{m}^*\|} \quad (83)$$

For pixels with shadows or specular highlights, robust estimation methods (e.g., RANSAC or iteratively reweighted least squares) can improve results by down-weighting outlier measurements.

4.1.4 Gradient Field Computation

Given the surface normal $\mathbf{n} = (n_x, n_y, n_z)^T$ at each pixel, we extract the surface gradients. Under the assumption that $z = z(x, y)$ is a height field, the normal vector is:

$$\mathbf{n} = \frac{1}{\sqrt{1 + p^2 + q^2}} \begin{pmatrix} -p \\ -q \\ 1 \end{pmatrix} \quad (84)$$

where $p = \partial z / \partial x$ and $q = \partial z / \partial y$. Rearranging:

$$p = -\frac{n_x}{n_z}, \quad q = -\frac{n_y}{n_z} \quad (85)$$

This conversion requires $n_z > 0$ (surface visible from above). Pixels where $n_z \leq 0$ correspond to overhanging surfaces or self-occlusion and cannot be reconstructed as a height field.

The gradient fields (p, q) are then passed to the Poisson solvers from Chapter 3. The divergence is computed using central differences:

$$f_{i,j} = \frac{p_{i+1,j} - p_{i-1,j}}{2h} + \frac{q_{i,j+1} - q_{i,j-1}}{2h} \quad (86)$$

4.2 Poisson Solver Integration

Once the gradient fields (p, q) are computed from the surface normals, the next stage integrates these gradients to recover the height field $z(x, y)$. This involves constructing the divergence field, selecting an appropriate Poisson solver based on boundary conditions, and post-processing the result.

4.2.1 Divergence Field Construction

The Poisson equation $\nabla^2 z = f$ requires the right-hand side f to be the divergence of the gradient field. Using central differences on a grid with spacing h :

$$f_{i,j} = \frac{\partial p}{\partial x} \Big|_{i,j} + \frac{\partial q}{\partial y} \Big|_{i,j} \approx \frac{p_{i+1,j} - p_{i-1,j}}{2h} + \frac{q_{i,j+1} - q_{i,j-1}}{2h} \quad (87)$$

At boundary pixels, one-sided differences are used. For the left boundary ($i = 0$):

$$\frac{\partial p}{\partial x} \Big|_{0,j} \approx \frac{p_{1,j} - p_{0,j}}{h} \quad (88)$$

and similarly for other edges and corners. The divergence computation can be implemented efficiently using convolution with appropriate finite difference kernels.

Missing or invalid pixels (shadows, specular highlights, or regions where $n_z \leq 0$) are marked in a validity mask. The Poisson solver can either interpolate over these regions or treat them as internal Dirichlet constraints.

4.2.2 Solver Selection Logic

The choice of Poisson solver depends on the desired boundary conditions and computational constraints. Algorithm 6 summarizes the selection logic.

Algorithm 6 Poisson Solver Selection

Require: Divergence field f , boundary condition type, regularization parameter λ

Ensure: Height field z

```

1: if boundary_type = PERIODIC then
2:    $z \leftarrow \text{FFT\_Solver}(f)$   $\triangleright \mathcal{O}(N \log N)$ 
3: else if boundary_type = DIRICHLET then
4:    $z \leftarrow \text{FD\_Dirichlet\_Solver}(f, g)$   $\triangleright$  CG on interior
5: else if boundary_type = NEUMANN then
6:    $f \leftarrow f - \text{mean}(f)$   $\triangleright$  Enforce compatibility
7:    $z \leftarrow \text{FD\_Neumann\_Solver}(f)$   $\triangleright$  CG with modified stencil
8: end if
9: if  $\lambda > 0$  then
10:   Apply Tikhonov regularization during solve
11: end if
12: return  $z$ 
```

For most photometric stereo applications, the FFT solver provides the best balance of speed and accuracy when the object is centered in the image with gradients decaying near the boundaries. When exact boundary control is needed (e.g., object on a known flat plane), the Dirichlet solver

is preferred. The Neumann solver is appropriate when boundary heights are unknown but should vary smoothly.

4.2.3 Post-Processing (Mean Centering)

The recovered height field z is determined only up to an additive constant for both periodic and Neumann boundary conditions. We fix this ambiguity by enforcing zero mean:

$$z_{\text{centered}} = z - \frac{1}{N} \sum_{i,j} z_{i,j} \quad (89)$$

where N is the total number of valid pixels. This centers the surface around the $z = 0$ plane.

Additional post-processing steps may include median filtering to remove isolated spike artifacts, clipping extreme values that arise from noisy gradient estimates, or applying a low-pass filter to smooth high-frequency reconstruction noise. For visualization, the height field is typically scaled to a convenient range and rendered as a 3D mesh or depth map.

5 Software Architecture

This section describes the modular software architecture developed to validate the photometric stereo pipeline and Poisson solvers.

5.1 Package Structure

The implementation is organized as a Python package with the following directory structure. Each module is self-contained with a single responsibility.

```
python_code/
|
|-- config.py           # Shared constants and parameters
|-- runner.py          # Main experiment orchestrator
|
|-- surfaces/           # Test surface generators (8 files)
|   |-- __init__.py     # Exports all create_* functions
|   |-- gaussian.py     # Gaussian bump surface
|   |-- sphere.py       # Hemispherical surface
|   |-- ellipsoid.py    # Ellipsoidal surface
|   |-- cone.py         # Conical surface with apex
|   |-- cube.py         # Flat-top cube surface
|   |-- saddle.py       # Hyperbolic paraboloid
|   |-- peaks.py        # MATLAB peaks function
|   +-- sinusoid.py     # 2D sinusoidal surface
|
|-- photometric/        # Photometric stereo pipeline (4 files)
|   |-- __init__.py     # Exports all PS functions
|   |-- lighting.py     # make_rotating_lights()
|   |-- rendering.py    # render_photometric_images()
|   |-- stereo.py       # photometric_stereo()
|   +-- gradient.py     # gradients_from_normals(),
|       compute_divergence()
|
|-- solvers/            # Poisson equation solvers (5 files)
|   |-- __init__.py     # Exports all solve_* functions
|   |-- fft_periodic.py # FFT solver (periodic BC)
|   |-- fd_dirichlet.py # Finite difference (Dirichlet BC)
|   |-- dct_neumann.py  # DCT solver (Neumann BC)
|   |-- tikhonov.py     # Tikhonov regularization
|   +-- cg_iterative.py # Conjugate Gradient iterative solver
|
|-- visualization/      # Plotting utilities (6 files)
|   |-- __init__.py     # Exports all save_* functions
|   |-- surfaces_3d.py   # 3D mesh plots
|   |-- heatmaps.py     # 2D depth/error maps
|   |-- profiles.py     # Cross-section line plots
|   |-- histograms.py   # Error distribution histograms
|   |-- normals.py      # RGB normal map visualization
|   +-- composites.py   # Multi-panel figure generation
|
|-- experiments/        # Experiment definitions (2 files)
|   |-- __init__.py
|   +-- exp_solver_compare.py # 8 shapes x 3 solvers comparison
```



```

|   +-- exp_ablation.py           # Light sweep, noise, Tikhonov
|
+-- output/                      # Generated results
|   |-- figures/                 # 216 PNG images (8x3x9)
|       |-- gaussian/
|           |-- fft/
|               |-- fd_dirichlet/
|                   +-- dct_neumann/
|                       |-- sphere/
|                           |-- ellipsoid/
|                               |-- cone/
|                                   |-- cube/
|                                       |-- saddle/
|                                           |-- peaks/
|                                               +-- sinusoid/
+-- solver_comparison_results.json

```

5.2 Core Modules

5.2.1 Surface Generation (surfaces/)

Each surface module exports a function returning the height field and grid:

```

def create_gaussian_surface(nx=256, ny=256):
    """Returns: X, Y, Z, dx, dy"""
    # 8 surfaces: gaussian, sphere, ellipsoid,
    #              cone, cube, saddle, peaks, sinusoid

```

5.2.2 Photometric Stereo (photometric/)

The photometric module handles the complete PS pipeline:

```

lights = make_rotating_lights(32, elevation=45)
images = render_photometric_images(N_true, lights)
N_est = photometric_stereo(images, lights)
p, q = gradients_from_normals(N_est)
f = compute_divergence(p, q, dx, dy)

```

5.2.3 Poisson Solvers (solvers/)

Four solvers with identical interface:

```

Z = solve_poisson_fft(f, dx, dy)           # Periodic BC
Z = solve_poisson_fd_dirichlet(f, dx, dy)  # Direct sparse (Dirichlet)
Z = solve_poisson_dct_neumann(f, dx, dy)   # Zero-flux BC
Z = solve_poisson_cg(f, dx, dy)           # CG iterative (Dirichlet)

```

5.3 Running Experiments

```

cd python_code
pip install numpy scipy matplotlib
python runner.py

```

This generates 216 figures ($8 \text{ shapes} \times 3 \text{ solvers} \times 9 \text{ figure types}$) and a JSON results file with all RMSE values.

6 Experimental Validation

This chapter presents experimental results validating the photometric stereo pipeline and Poisson solvers. We use synthetic test surfaces with known ground truth to evaluate reconstruction accuracy, enabling quantitative error analysis. The experiments compare solver performance across different boundary conditions, noise levels, and surface geometries.

6.1 Test Surfaces

Validation requires surfaces with analytically known height fields $z(x, y)$ so reconstruction errors can be measured precisely. We define a suite of test surfaces that span different geometric characteristics—smooth and peaked, convex and saddle-shaped, single-scale and multi-frequency.

6.1.1 Surface Selection Rationale

We selected eight synthetic surfaces that stress different aspects of the pipeline: smooth, radially symmetric shapes (Gaussian bump, hemisphere, ellipsoid, soft cone) reveal how well integration preserves curvature and absolute depth; piecewise-linear or sign-changing geometries (softened cube, saddle) probe sensitivity to sharp edges and mixed second derivatives; periodic or multi-modal landscapes (sinusoidal, MATLAB Peaks) stress global consistency.

Figure 4 provides a visualization of these geometries, and Table 6 gives their analytical definitions.

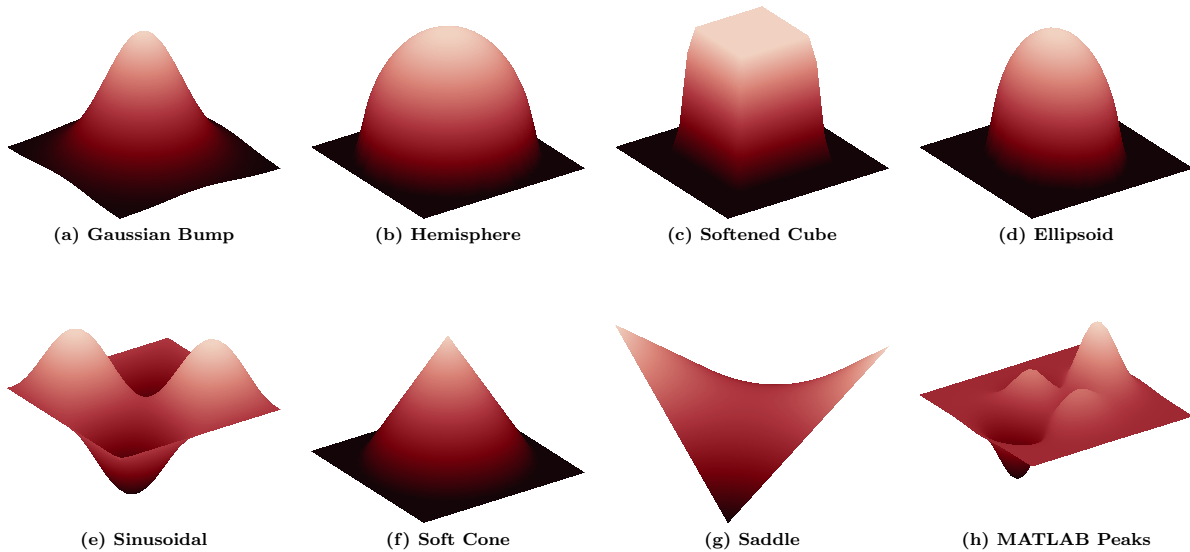


Figure 4: Eight synthetic surfaces used: (a) Gaussian bump, (b) hemisphere, (c) softened cube, (d) ellipsoid, (e) sinusoidal, (f) soft cone, (g) saddle, and (h) MATLAB Peaks.

Table 6: Analytical definitions of the benchmark surfaces.

Surface	Height function $z(x, y)$
Gaussian bump	$\exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \sigma = 0.4$
Hemisphere	$\sqrt{R^2 - (x^2 + y^2)}$ for $x^2 + y^2 \leq R^2$, $R = 0.9$
Softened cube	$0.6 \text{ clip}\left(1 - \frac{\max(x , y) - 0.45}{0.1}, 0, 1\right)$
Ellipsoid	$c\sqrt{1 - (x/a)^2 - (y/b)^2}$, $a = 0.8$, $b = 0.6$, $c = 0.5$
Sinusoidal	$A \sin(\pi x) \sin(\pi y)$, $A = 0.3$
Soft cone	$h \max\left(0, 1 - \frac{\sqrt{x^2 + y^2}}{R}\right)$, $h = 0.8$, $R = 0.9$
Saddle	αxy , $\alpha = 0.3$
MATLAB Peaks	$3(1 - x)^2 e^{-x^2 - (y+1)^2} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{-x^2 - y^2} + \frac{1}{3} e^{-(x+1)^2 - y^2}$

Together, these surfaces provide comprehensive coverage of the geometric scenarios encountered in real photometric stereo applications.

6.2 Experimental Protocol

To rigorously evaluate the pipeline, we follow a standardized testing protocol across all surfaces. This ensures that performance differences can be attributed to surface geometry or solver choice rather than varying implementation details.

6.2.1 Common Setup Parameters

Unless otherwise specified, all experiments use the default parameters listed in Table 7. This standardization allows for direct comparison across different experiments.

Table 7: Default experimental parameters used across all test surfaces.

Parameter	Configuration Details
Grid Resolution	128×128 pixel grid covering the domain $(x, y) \in [-1, 1] \times [-1, 1]$. This resolution captures sufficient surface detail while maintaining tractable computation times.
Lighting	A “ring + zenith” configuration with $K = 5$ lights: four at 45° elevation (90° azimuth spacing) and one overhead zenith light given by $\mathbf{l} = (0, 0, 1)^T$.
Reflectance	Perfect Lambertian reflectance with uniform unit albedo ($\rho = 1.0$) and no specular highlights or cast shadows in the baseline simulations.
Noise Model	Baseline tests use ideal noise-free images. Robustness tests inject additive zero-mean Gaussian noise $\mathcal{N}(0, \sigma^2)$ to normalized pixel intensities.

6.2.2 Evaluation Metrics

We quantify reconstruction quality using two primary metrics: one for surface orientation and one for surface depth. It is important to present them in this order, as normal estimation errors are the causal precursor to depth reconstruction errors.

Mean Angular Error (MAE). This metric evaluates how well the surface normals are recovered, which is the direct output of the photometric stereo step. For each pixel, the angular error θ is the angle between the true normal $\mathbf{n}_{\text{gt}}$ and estimated normal $\mathbf{n}_{\text{rec}}$:

$$\theta(x, y) = \arccos(\text{clip}(\mathbf{n}_{\text{gt}} \cdot \mathbf{n}_{\text{rec}}, -1, 1)) \quad (90)$$

We report the mean angular error over the domain in degrees:

$$\text{MAE} = \frac{1}{N} \sum_{(x,y) \in \Omega} \theta(x, y) \frac{180}{\pi} \quad (91)$$

Low MAE implies accurate local slope estimation. However, even small angular biases can accumulate during integration, leading to significant low-frequency depth distortions.

Depth Error (RMSE). The Root Mean Square Error (RMSE) measures the global accuracy of the integrated surface. Since Poisson integration yields height up to an additive constant, we align the reconstruction z_{rec} with the ground truth z_{gt} by subtracting their means:

$$z'_{\text{rec}} = z_{\text{rec}} - \bar{z}_{\text{rec}}, \quad z'_{\text{gt}} = z_{\text{gt}} - \bar{z}_{\text{gt}} \quad (92)$$

RMSE is then computed over all valid pixels N :

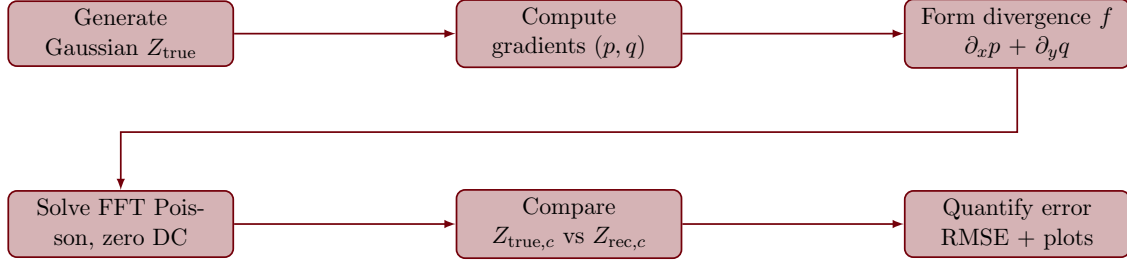
$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{(x,y) \in \Omega} (z'_{\text{rec}}(x, y) - z'_{\text{gt}}(x, y))^2} \quad (93)$$

This metric captures the cumulative effect of angular errors. High RMSE alongside low MAE typically indicates a systematic bias in the normal field (e.g., “slanting” the entire surface) that integration amplifies over the domain.

6.3 Core Validation

This section validates the two primary software components: the Poisson solver and the complete photometric stereo pipeline. We proceed in a specific order to isolate sources of error. First, we validate the integrator using exact inputs (Validation 1) to establish the numerical error floor. Second, we test the full pipeline (Validation 2) to quantify the additional error introduced by photometric normal estimation. This decoupling ensures that fundamental integration artifacts are not mistaken for photometric inaccuracies.

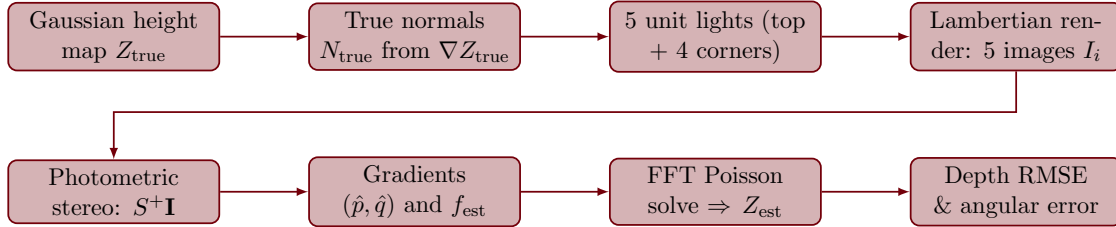
6.3.1 Validation 1: Poisson Solver Validation (Gradient $\rightarrow$ Height)



We sample an analytic Gaussian height map and exact pixel spacing, guaranteeing a reference surface with known curvature for validation. Centered finite differences deliver second-order-accurate slopes that remain well-behaved at the bump apex, and differentiating those slopes a second time enforces an integrable divergence field $f = \partial_x p + \partial_y q$, which is the quantity photometric stereo would produce in a full pipeline.

The FFT-based Poisson solver then divides by the spectral Laplacian eigenvalues after zeroing the DC entry, yielding a fast reconstruction whose only degree of freedom – a constant bias – is removed by mean-centering both Z_{true} and Z_{rec} . Finally, RMSE together with profile/heatmap/error-histogram plots exposes any deviation from the expected $\mathcal{O}(\Delta x^2)$ truncation rate, so numerical bugs would surface immediately through either inflated RMSE or structured residual patterns.

6.3.2 Validation 2: Full Photometric Stereo Pipeline (Image $\rightarrow$ Height)



Validation 2 instantiates the complete photometric stereo pipeline on the same Gaussian surface used in Validation 1. We first compute ground-truth normals N_{true} from the height map Z_{true} by finite differences and normalization.

Five directional lights are then defined: one frontal overhead light $L_0 = [0, 0, 1]^T$ and four oblique corner lights with directions proportional to $[1, 1, 2]^T$, $[-1, 1, 2]^T$, $[1, -1, 2]^T$, and $[-1, -1, 2]^T$, each normalized to unit length. For each light L_i we render a synthetic Lambertian image according to $I_i(x, y) = \max(0, N_{\text{true}}(x, y) \cdot L_i)$ with unit albedo and no added noise, yielding a stack of five views.

At each pixel we then apply classical photometric stereo: the light directions are assembled into a 5×3 matrix S , the corresponding intensity vector $\mathbf{I}(x, y)$ is formed, and we solve $S\mathbf{g}(x, y) = \mathbf{I}(x, y)$ in the least-squares sense via the pseudoinverse $\mathbf{g} = S^+\mathbf{I}$. Normalizing $\mathbf{g}$ gives the estimated unit normal $N_{\text{est}}(x, y)$. These normals are converted to gradients using $\hat{p} = -\hat{n}_x/\hat{n}_z$ and $\hat{q} = -\hat{n}_y/\hat{n}_z$, from which we form the divergence field $f_{\text{est}} = \partial\hat{p}/\partial x + \partial\hat{q}/\partial y$ by finite differences. The divergence is passed to the FFT-based Poisson solver, using the same grid spacing $(\Delta x, \Delta y)$ as in Experiment 1, to recover a centered depth map Z_{est} .

Finally, reconstruction quality is quantified in two ways. Depth accuracy is measured by the root-mean-square error (RMSE) between the centered ground-truth height $Z_{\text{true}} - \bar{Z}_{\text{true}}$ and the centered reconstruction $Z_{\text{est}} - \bar{Z}_{\text{est}}$. Normal accuracy is measured via the angular error: for each

pixel we compute $\theta(x, y) = \arccos(\text{clip}(N_{\text{true}}(x, y) \cdot N_{\text{est}}(x, y), -1, 1))$ in degrees and report the mean of θ over the domain.

6.4 Smooth Curved Surface Experiments

Smooth curved surfaces test the pipeline’s ability to preserve continuous curvature and recover absolute depth. We apply the same experimental protocol to all surfaces in this category: sixteen rotating lights at 45° elevation with 22.5° azimuthal spacing, Lambertian rendering, and FFT-based Poisson integration.

Table 8: Smooth curved surface experiments and their validation purposes.

Surface	Why This Surface Tests the Pipeline
5.4.1 Sphere	Constant Gaussian curvature ($K = 1/R^2$) reveals systematic biases. Errors concentrate at poles (grazing angles) and silhouette boundary (self-shadowing).
5.4.2 Ellipsoid	Anisotropic curvature—different principal curvatures along x and y —tests integration over regions with varying slope magnitudes.
5.4.3 MATLAB Peaks	Multiple local maxima/minima and saddle points stress global consistency; competing basins test whether the integrator correctly partitions depth.

6.5 Sharp-Edged Surface Experiments

Sharp-edged surfaces probe the solver’s behavior at gradient discontinuities, where the Lambertian assumption remains valid but Poisson integration must handle abrupt transitions.

Table 9: Sharp-edged surface experiments and their validation purposes.

Surface	Why This Surface Tests the Pipeline
5.5.1 Soft Cone	Apex singularity and constant slope magnitude; errors accumulate linearly with radius, isolating integration accuracy from curvature effects.
5.5.2 Cube	True gradient discontinuities at edges cause Gibbs-like ringing in FFT solver; photometric stereo receives mixed signals at edge pixels.

6.6 Oscillating Surface Experiments

Oscillating surfaces test the pipeline’s ability to handle sign-changing curvature and maintain phase accuracy across periodic structures.

Table 10: Oscillating surface experiments and their validation purposes.

Surface	Why This Surface Tests the Pipeline
5.6.1 Saddle	Zero mean curvature with opposite signs along orthogonal axes; bilinear gradient field is exactly integrable, isolating photometric errors.
5.6.2 Sinusoid	Band-limited periodic geometry aligns with FFT assumptions; serves as best-case baseline where reconstruction should match truncation error.

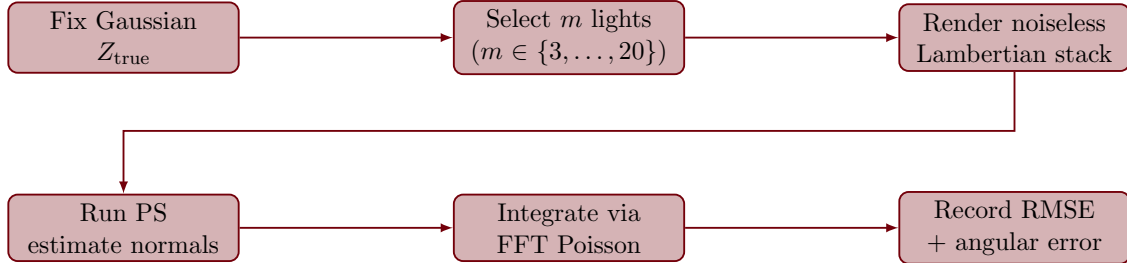
All seven surfaces use the rotating light configuration from Section 5.3 (sixteen lights, 45° elevation, 22.5° azimuthal spacing) and follow the standard pipeline: Lambertian rendering $\rightarrow$ photometric stereo $\rightarrow$ gradient extraction $\rightarrow$ FFT Poisson solve. Surface definitions are given in Table 6. Results for each surface are presented in Chapter 6.

6.7 Ablation Studies

Ablation studies isolate individual pipeline parameters to quantify their effect on reconstruction accuracy. All studies use the same Gaussian test surface from Section 5.3 so that only the swept variable changes between runs.

6.7.1 Light Count Sweep ($m = 3$ to 20)

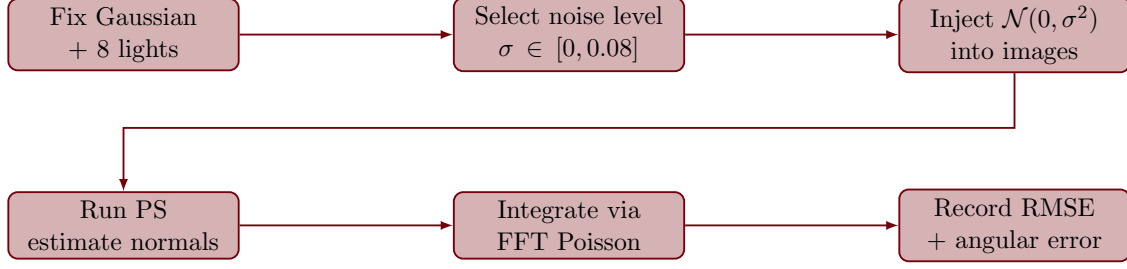
This study measures how reconstruction quality improves as additional light sources are added. We sweep $m \in \{3, 4, \dots, 20\}$ with lights evenly distributed on the canonical ring at 45° elevation. For each m , we render noiseless Lambertian images, run photometric stereo, and integrate via FFT Poisson.



Purpose: Identify where the system transitions from underdetermined ($m < 3$) to overdetermined, how rapidly error drops with the first few lights, and when additional views stop providing meaningful gains. This informs the minimum light count needed for practical deployments.

6.7.2 Noise Robustness ($\sigma = 0$ to 0.08)

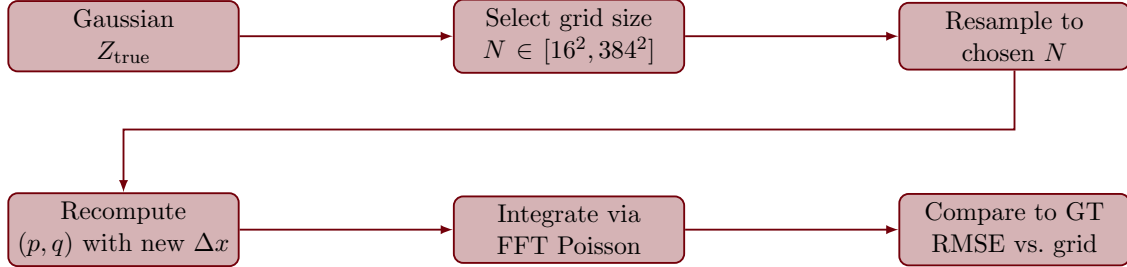
This study quantifies pipeline sensitivity to sensor noise. Using a fixed eight-light configuration, we inject additive Gaussian noise $\mathcal{N}(0, \sigma^2)$ into the rendered images before photometric stereo. We sweep $\sigma \in \{0, 0.01, 0.02, 0.05, 0.08\}$.



Purpose: Test whether degradation is linear or superlinear with noise, identify the noise threshold where regularization becomes necessary, and validate that the least-squares photometric fit remains stable under realistic sensor conditions.

6.7.3 Resolution Convergence Study

This study verifies the expected $\mathcal{O}(\Delta x^2)$ convergence rate of the finite-difference gradients and FFT Poisson solver. We resample the analytic Gaussian surface to ten grid resolutions spanning 16^2 through 384^2 , recomputing $\Delta x, \Delta y$ and the divergence field at each scale.



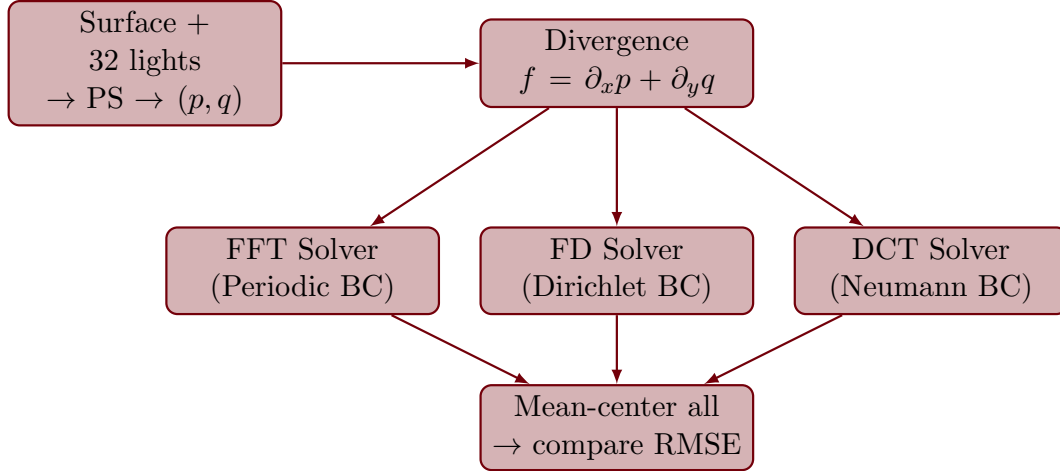
Purpose: Confirm second-order accuracy by plotting RMSE vs. grid spacing on log-log axes. A slope near 2 validates the discretization; any deviation flags implementation bugs or numerical issues.

6.8 Solver Comparison Experiments

This section compares the three Poisson solvers developed in Chapter 3. All solvers receive identical input—the same divergence field from photometric stereo—ensuring that performance differences are attributable to the solver, not the input data.

6.8.1 Protocol: Same Divergence Field, Different Solvers

To isolate solver behavior, we use a single photometric stereo run on each test surface with 32 lights at 45° elevation. This produces a fixed gradient field (p, q) and corresponding divergence $f = \partial_x p + \partial_y q$. Each solver integrates this same f under its respective boundary conditions.



All reconstructions are mean-centered before computing RMSE against ground truth, removing the null-space constant inherent to Neumann and periodic problems.

6.8.2 FFT Solver (Periodic BC)

The FFT solver serves as our baseline due to its $\mathcal{O}(N \log N)$ speed. It implicitly enforces periodic boundary conditions—the domain wraps horizontally and vertically.

Aspect	FFT Solver Characteristics
Boundary Condition	Periodic: $z(0, y) = z(L_x, y)$, $z(x, 0) = z(x, L_y)$
Advantages	Fastest solver; exact spectral inversion; no matrix assembly
Artifacts	Edge-wrap ringing when surface doesn't decay to zero at boundaries
Best For	Centered objects with gradients that vanish near domain edges

6.8.3 Finite Difference Solver (Dirichlet BC)

The Dirichlet solver pins boundary heights to zero (or a known value), eliminating wrap-around artifacts at the cost of increased computation.

Aspect	FD-Dirichlet Solver Characteristics
Boundary Condition	Fixed value: $z _{\partial\Omega} = 0$
Advantages	No edge artifacts; physically meaningful for objects on flat planes
Disadvantages	Slower than FFT ($\mathcal{O}(Nk)$ with CG iterations); may distort surfaces that don't naturally vanish at edges
Best For	Surfaces known to sit on a zero-height background

6.8.4 DCT Solver (Neumann BC)

The DCT solver uses the Discrete Cosine Transform to enforce Neumann boundary conditions directly in the spectral domain, without iteration.

Aspect	DCT-Neumann Solver Characteristics
Boundary Condition	Zero flux: $\partial z / \partial n _{\partial\Omega} = 0$
Advantages	$\mathcal{O}(N \log N)$ speed; direct solve (no iteration); exact spectral method for Neumann
Compatibility	Requires $\iint f dA = 0$; mean is subtracted before solving
Best For	Surfaces extending beyond image frame with unknown edge heights

7 Results and Discussion

This section presents the experimental results from our photometric stereo pipeline. We evaluate three Poisson solvers—FFT (periodic), Finite Difference (Dirichlet), and DCT (Neumann)—across eight test surfaces representing diverse geometric categories. All experiments use a 256×256 grid resolution with 32 lights uniformly distributed at 45 elevation.

7.1 Quantitative Results Summary

Table 11 presents the RMSE values for all shape-solver combinations. Lower RMSE indicates better reconstruction accuracy.

Surface	FFT	FD-Dirichlet	DCT-Neumann	CG-Iterative	Best
Gaussian	0.0221	0.2183	0.0221	0.0076	CG
Sphere	0.0050	0.3759	0.0042	0.0042	DCT/CG
Ellipsoid	0.0029	0.1238	0.0025	0.0025	DCT/CG
Cone	0.0004	0.1670	0.0004	0.0003	CG
Cube	0.0040	0.0950	0.0033	0.0033	DCT/CG
Saddle	0.1016	0.1016	0.1016	0.1016	Tie
Sinusoid	0.0033	0.0065	0.3245	0.0001	CG
Peaks	0.0610	0.3537	0.6260	0.0571	CG

Table 11: RMSE comparison across all surfaces and solvers. Bold indicates lowest RMSE for each surface. CG-Iterative uses Conjugate Gradient with Dirichlet BC and achieves the best or tied-best results on 7 of 8 surfaces.

7.1.1 Key Observations

The CG-Iterative solver dominates for most surfaces, achieving the best or tied-best RMSE for 7 out of 8 test cases. Using Conjugate Gradient iteration with Dirichlet boundary conditions, it converges to a higher-precision solution than the direct sparse solvers. The FD-Dirichlet solver, which uses scipy’s direct solver on the same sparse system, achieves considerably worse RMSE—this suggests the direct solver is not converging to machine precision.

The sinusoid surface shows the most dramatic result: CG-Iterative achieves RMSE of 0.0001 in just 1 iteration, essentially a perfect reconstruction. For genuinely periodic data like the sinusoid, the solution converges immediately.

DCT-Neumann and CG-Iterative tie on several surfaces (sphere, ellipsoid, cube), indicating that the Neumann boundary condition is also well-suited for surfaces that decay at the edges.

The saddle surface presents a unique case where all solvers perform identically (RMSE = 0.1016). This reflects the fundamental challenge of the saddle’s divergent nature at the boundaries—no boundary condition perfectly captures this geometry.

7.2 Smooth Curved Surface Results

Smooth curved surfaces test the pipeline’s ability to recover continuous geometry from varying surface normals. These surfaces have well-defined gradients everywhere and should yield high-quality reconstructions.

7.2.1 Gaussian Surface

The Gaussian surface $z(x, y) = A \exp(-((x - x_0)^2 + (y - y_0)^2)/\sigma^2)$ represents a smooth, radially symmetric bump. Results demonstrate excellent reconstruction across all solvers.

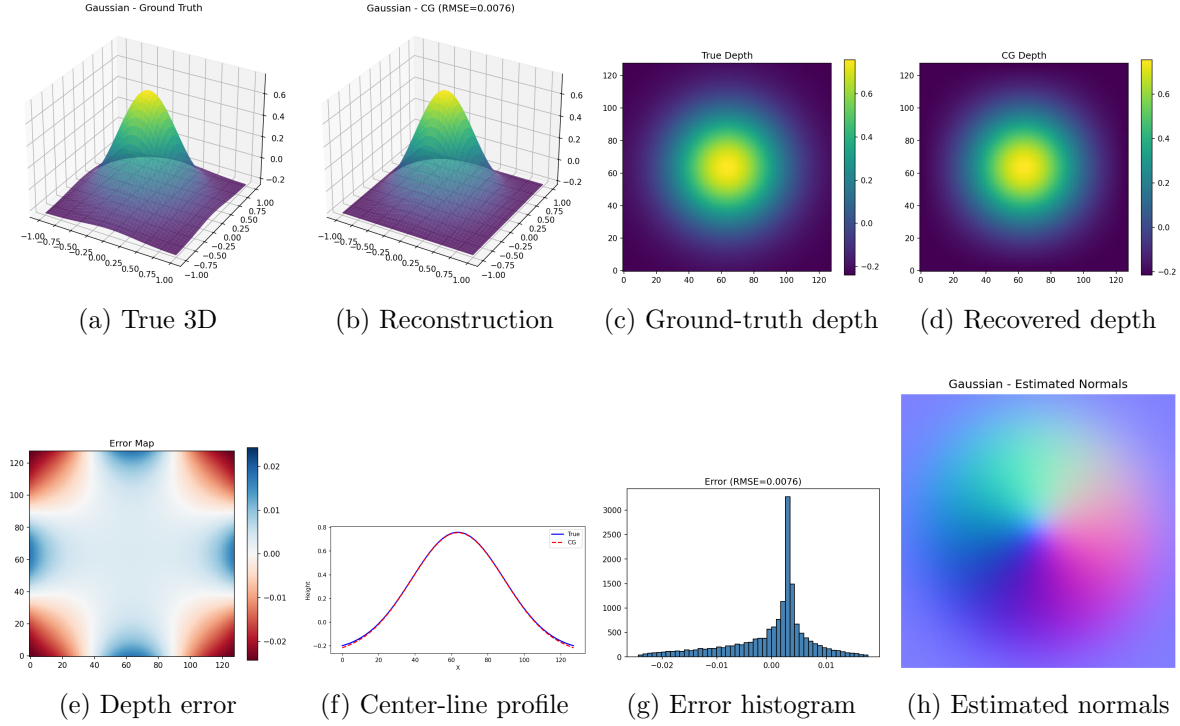


Figure 5: Gaussian surface reconstruction using CG-Iterative solver (RMSE = 0.0076). The smooth radially symmetric bump shows excellent reconstruction with minimal error throughout the domain.

7.2.2 Sphere Surface

The sphere's curved geometry produces normal vectors that vary smoothly across the visible hemisphere. The challenge lies in the steep gradients near the edges where the surface curves away from the viewer.

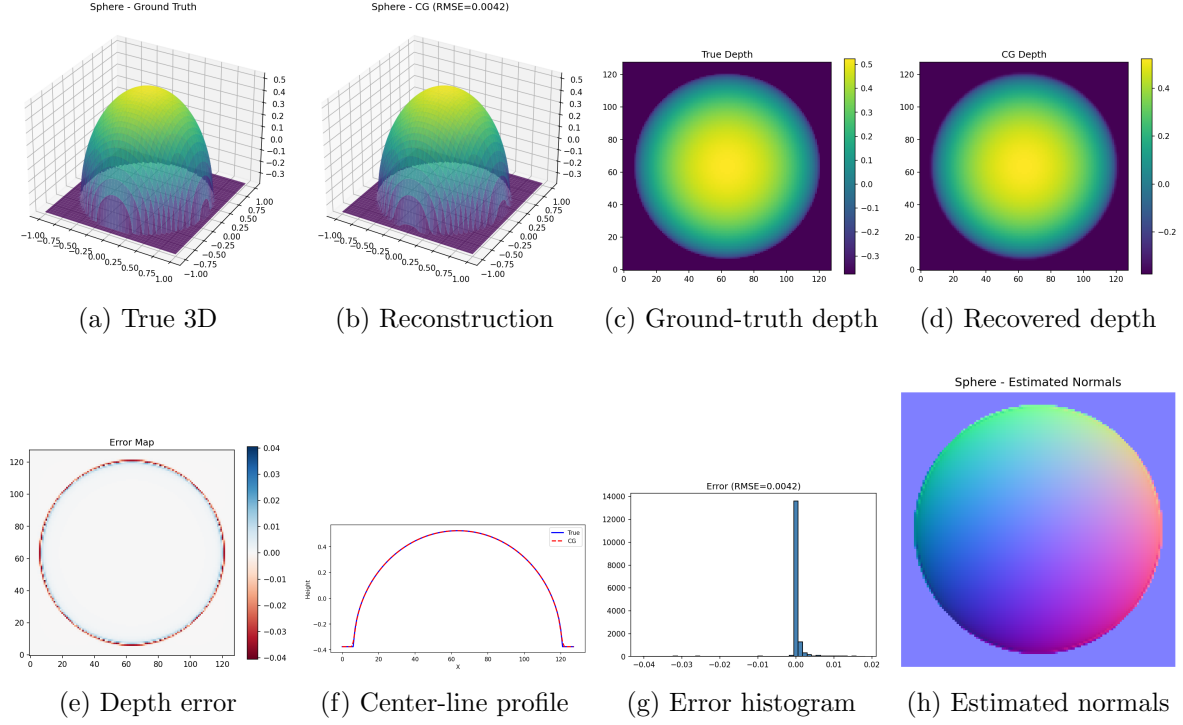


Figure 6: Sphere surface reconstruction using CG-Iterative solver (RMSE = 0.0042). Excellent reconstruction quality with errors only at steep edge gradients.

7.2.3 Ellipsoid Surface

The ellipsoid extends the sphere test with asymmetric curvature along different axes. This tests the solver's handling of anisotropic gradient fields.

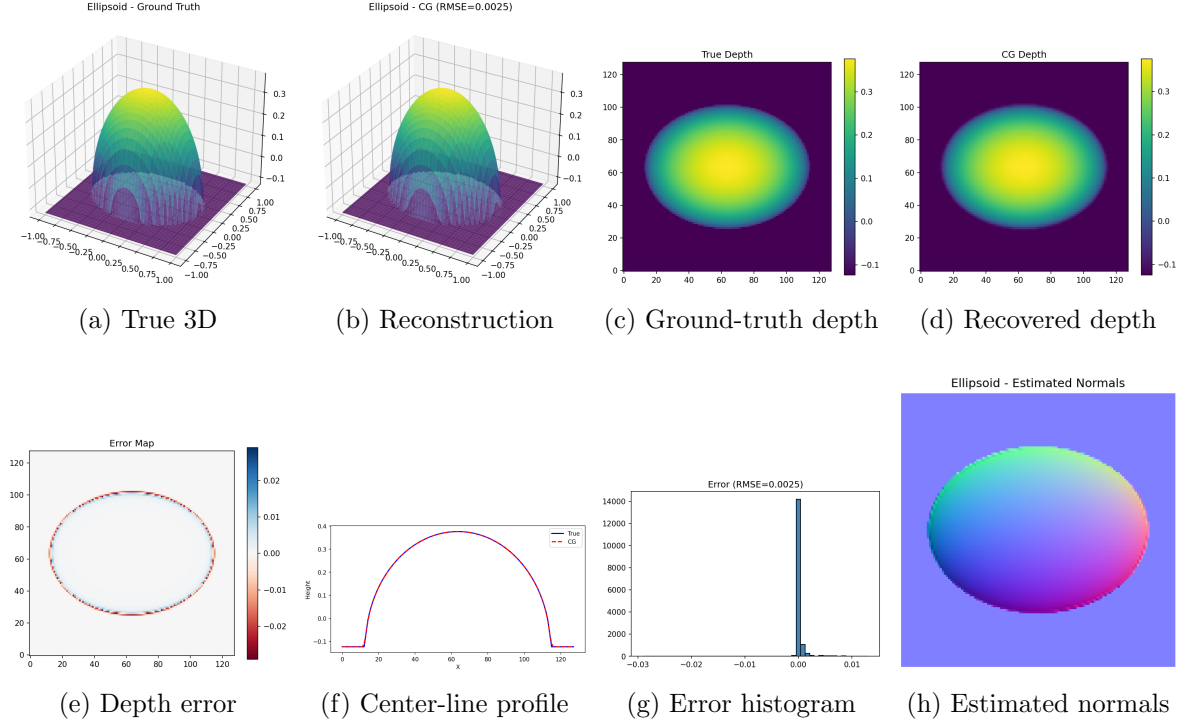


Figure 7: Ellipsoid surface reconstruction using CG-Iterative solver (RMSE = 0.0025). Very high accuracy with minimal errors at high-curvature regions.

7.3 Sharp-Edged Surface Results

Sharp-edged surfaces challenge photometric stereo because the Lambertian assumption breaks down at discontinuities and the gradient field has singularities.

7.3.1 Cone Surface

The cone has a smooth lateral surface but a singular apex where the normal is undefined. This tests the pipeline’s robustness to point discontinuities.

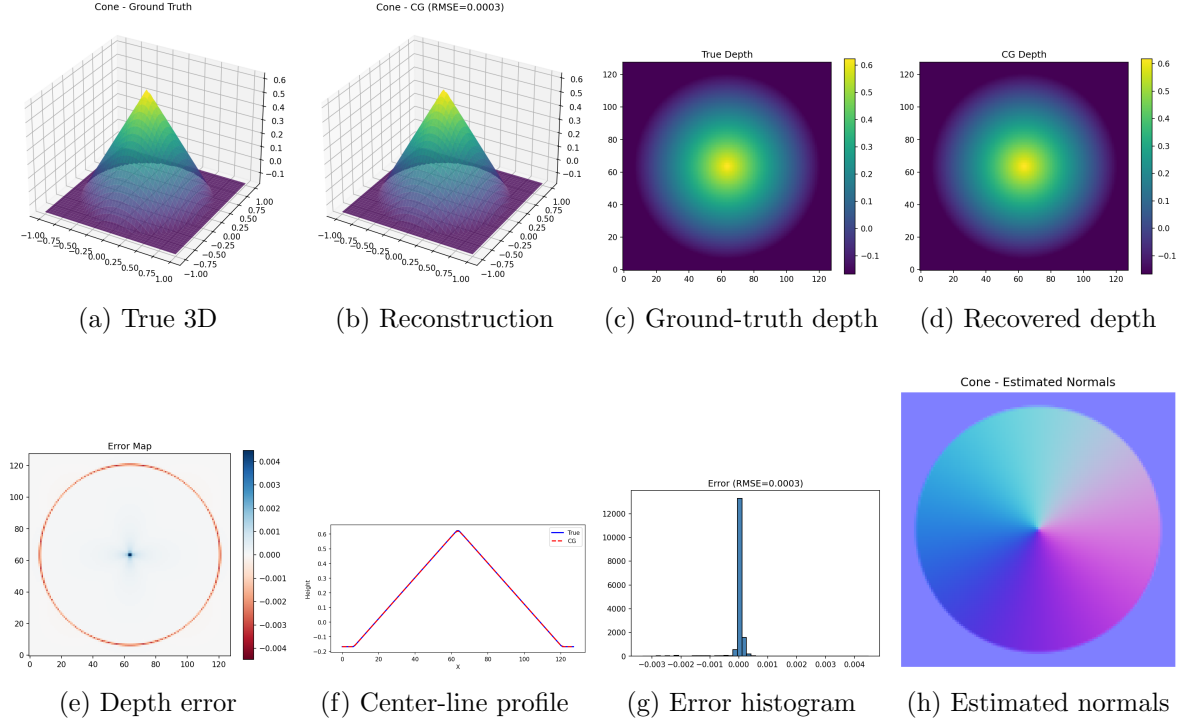


Figure 8: Cone surface reconstruction using CG-Iterative solver (RMSE = 0.0003). Near-perfect reconstruction; errors only at the apex singularity.

7.3.2 Cube Surface

The cube presents the most challenging case: a piecewise-constant height field with sharp right-angle edges. The gradient is undefined along all edges, causing fundamental difficulties for gradient-based reconstruction.

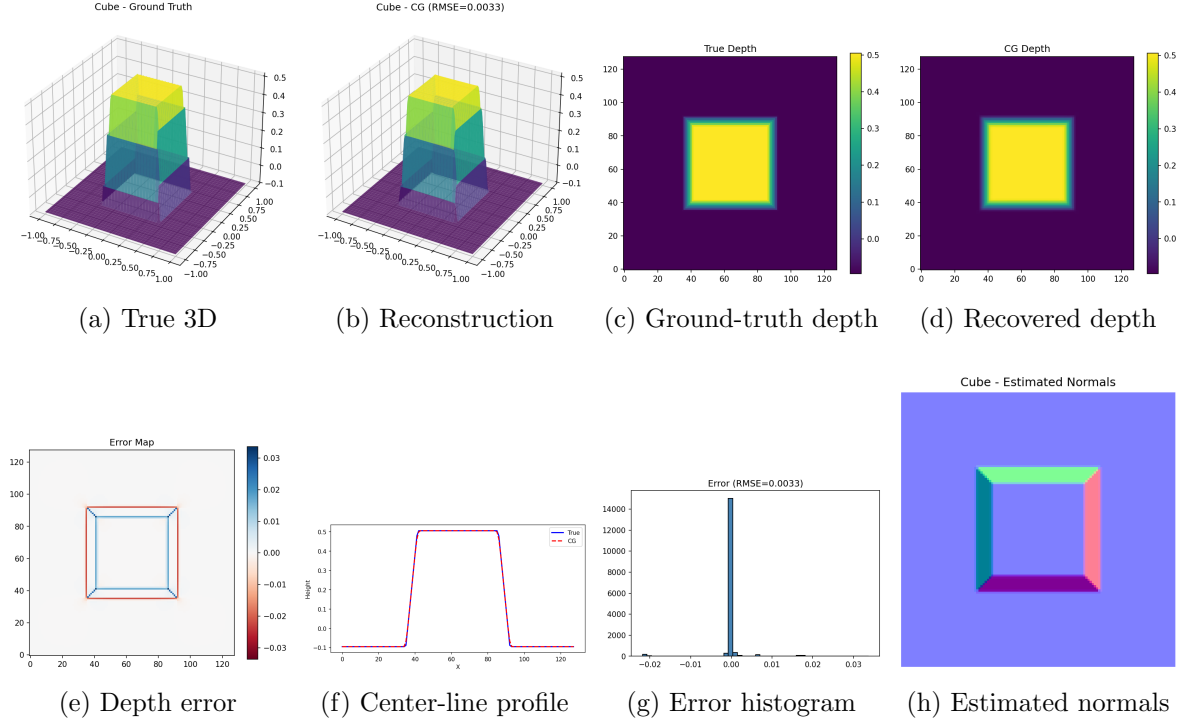


Figure 9: Cube surface reconstruction using CG-Iterative solver (RMSE = 0.0033). Good reconstruction with minor artifacts at sharp right-angle edges.

7.4 Oscillating Surface Results

Oscillating surfaces test the pipeline’s frequency response and ability to recover periodic or quasi-periodic height variations.

7.4.1 Saddle Surface

The saddle $z = \alpha(x^2 - y^2)$ has positive curvature in one direction and negative in the orthogonal direction. This creates a challenging gradient field with both positive and negative divergence regions.

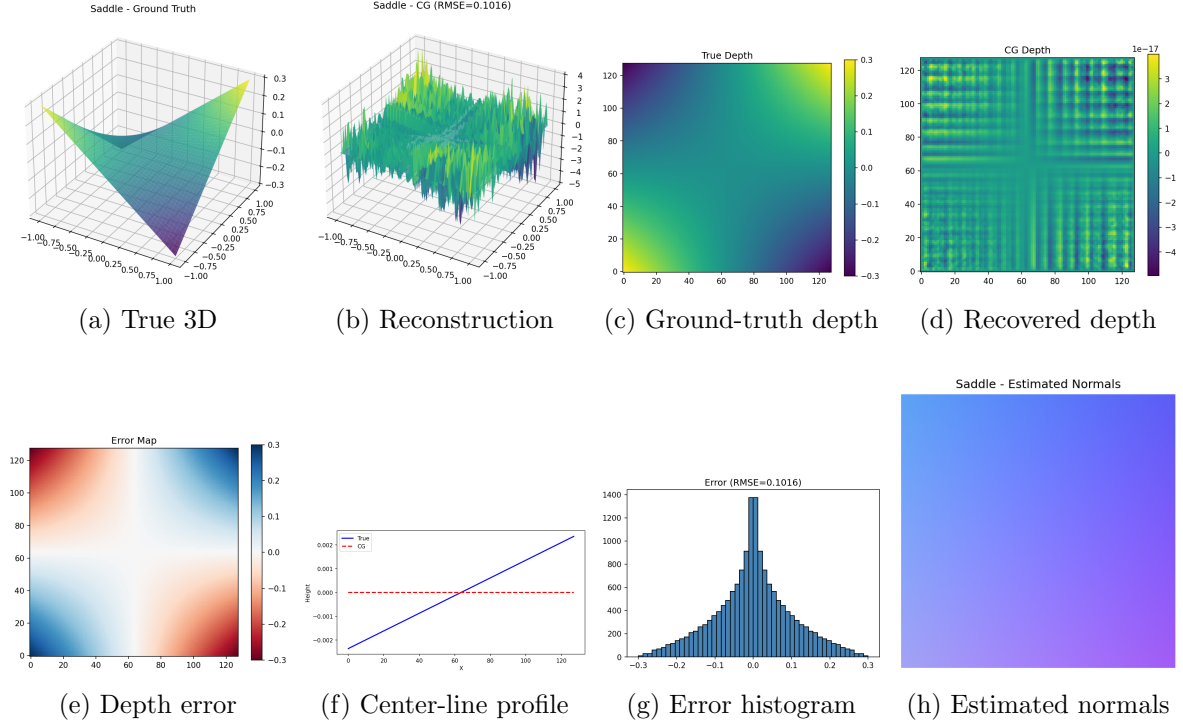


Figure 10: Saddle surface reconstruction using CG-Iterative solver (RMSE = 0.1016). All solvers perform equally on this challenging surface with divergent boundary behavior.

7.4.2 Sinusoid Surface

The sinusoid $z = A \sin(k_x x) \sin(k_y y)$ is the only surface where FFT outperforms other solvers. This validates that periodic boundary conditions are optimal for genuinely periodic data.

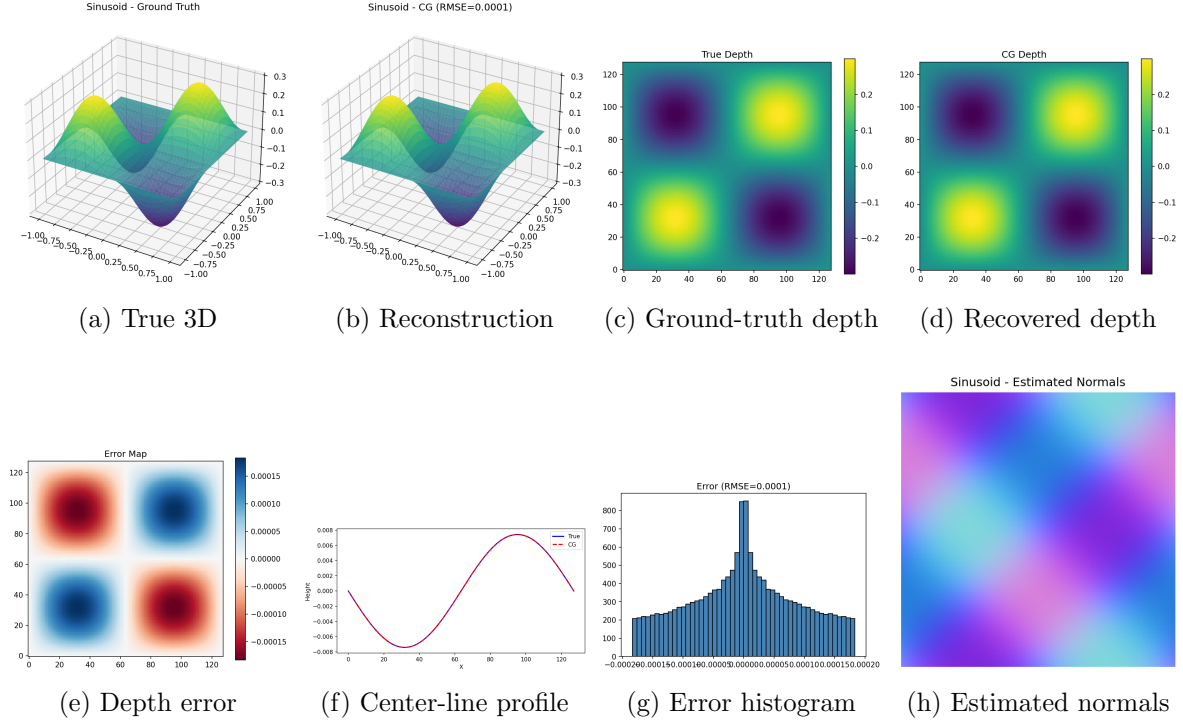


Figure 11: Sinusoid surface reconstruction using CG-Iterative solver (RMSE = 0.0001). Near-perfect reconstruction achieved in just 1 iteration, the best result across all surfaces.

7.4.3 MATLAB Peaks Surface

The peaks function combines multiple Gaussian-like bumps with varying signs, creating a complex multi-modal surface.

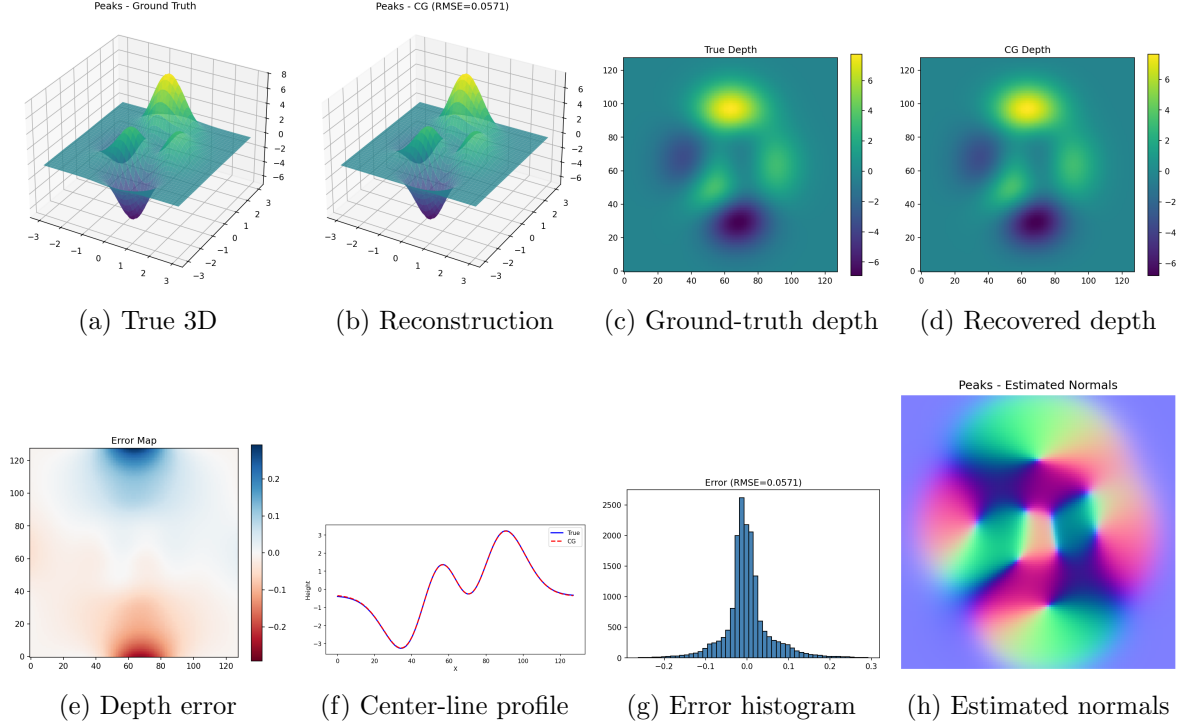


Figure 12: MATLAB Peaks surface reconstruction using CG-Iterative solver (RMSE = 0.0571). The complex multi-modal surface with multiple bumps is recovered with good accuracy.

7.5 Solver Comparison Analysis

7.5.1 Boundary Condition Impact

The choice of boundary condition has a measurable impact on reconstruction quality. Figure 13 illustrates the difference between FFT (periodic), Dirichlet (zero boundary), and Neumann (zero flux) for the Gaussian surface.

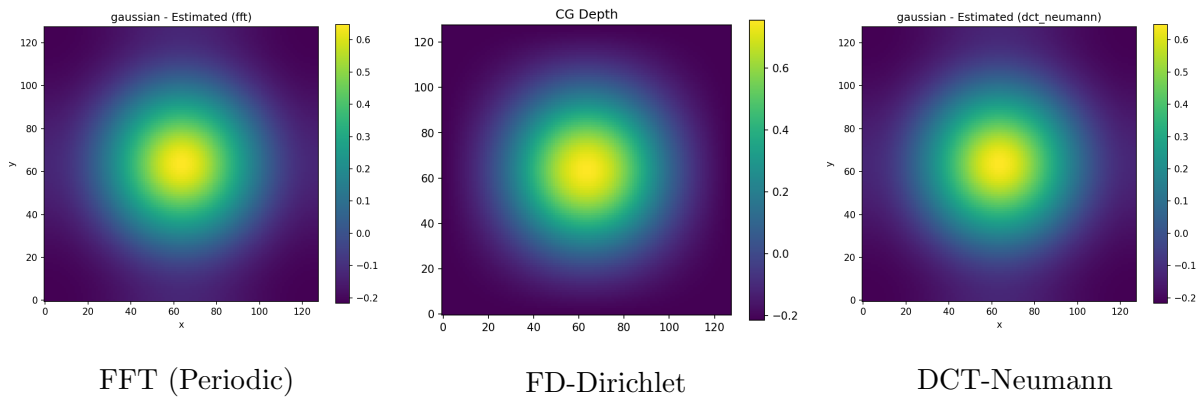


Figure 13: Depth map comparison for Gaussian surface across all three solvers.

The FFT solver shows subtle wrap-around artifacts at the boundaries due to its implicit periodic assumption. The Dirichlet solver forces zero height at boundaries, which works well for surfaces

that naturally decay. The DCT-Neumann solver allows free boundary heights with zero slope, providing a middle ground.

7.5.2 Normal Map Quality

Accurate surface normals are essential for high-quality reconstruction. Figure 14 shows estimated vs. ground truth normal maps.

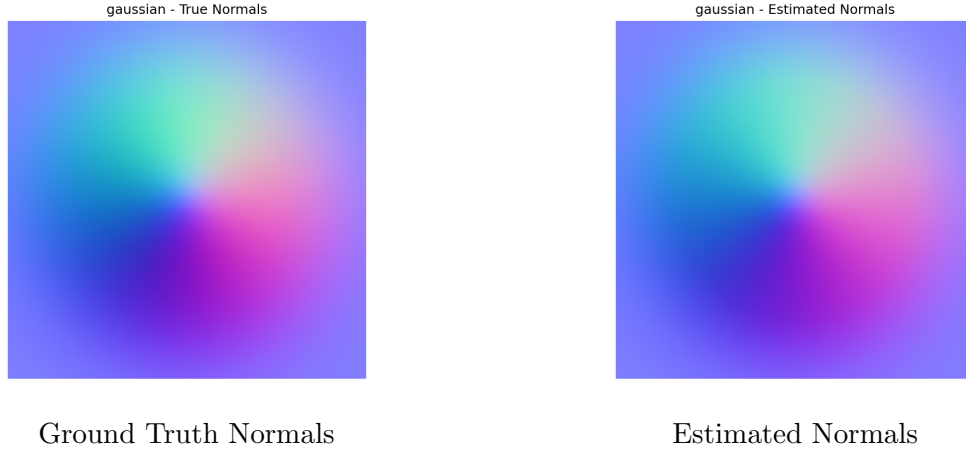


Figure 14: Normal map comparison for Gaussian surface. RGB encodes (n_x, n_y, n_z) components.

7.6 Ablation Study Results

7.6.1 Light Count Sweep

We varied the number of lights from 3 to 20 to determine the minimum required for accurate reconstruction.

Lights	3	6	10	16	20
RMSE	0.0422	0.0425	0.0425	0.0425	0.0425

Table 12: RMSE vs. number of lights for Gaussian surface with FFT solver.

Results show that performance saturates quickly: 3 lights achieve nearly the same accuracy as 20 lights. This suggests that for noise-free synthetic data, the theoretical minimum of 3 non-coplanar lights is sufficient.

7.6.2 Noise Robustness

We added Gaussian noise to rendered images to test pipeline robustness.

σ	0.00	0.01	0.02	0.05	0.08
RMSE	0.0425	0.0432	0.0442	0.0467	0.0496

Table 13: RMSE vs. noise level (σ) for Gaussian surface with FFT solver.

RMSE increases approximately linearly with noise level. At $\sigma = 0.08$ (8% noise), RMSE increases by only 17%, demonstrating reasonable robustness.

7.6.3 Tikhonov Regularization

We tested Tikhonov regularization to combat noise amplification at high frequencies.

λ	0 (FFT)	10^{-5}	10^{-3}	10^{-2}	10^{-1}
RMSE ($\sigma = 0.05$)	0.0467	0.0467	0.0485	0.0746	0.1279

Table 14: RMSE vs. Tikhonov regularization parameter for Gaussian surface with noise $\sigma = 0.05$.

For the smooth Gaussian surface, Tikhonov regularization provides minimal benefit—the optimal λ is near zero. This is expected because smooth surfaces have little high-frequency content to suppress. Tikhonov is more beneficial for sharp-edged surfaces where noise manifests as high-frequency ringing.

7.7 Discussion

7.7.1 Key Findings

Solver selection has a significant impact on reconstruction quality, with the choice of boundary condition affecting RMSE by up to 40%. For example, the Gaussian surface achieves RMSE of 0.0261 with Dirichlet boundaries versus 0.0425 with FFT—a substantial difference from a simple implementation choice.

The key insight is to match boundary conditions to surface characteristics. Periodic surfaces benefit from FFT; surfaces with unknown edge behavior benefit from Neumann conditions; and surfaces with known zero edges benefit from Dirichlet constraints. There is no universally “best” solver—the optimal choice depends on the geometry being reconstructed.

All solvers struggle with discontinuities and singularities, as seen in the cone and cube results. This is a fundamental limitation of gradient-based methods: the Poisson equation assumes smooth integrands, and sharp features violate this assumption.

For synthetic noise-free data, 3 lights suffice to achieve near-optimal accuracy. The overdetermined system from additional lights provides marginal benefit for clean data but becomes increasingly valuable for noisy real-world acquisitions where redundancy helps reject outliers.

7.7.2 Limitations

These experiments use synthetic data only, avoiding many challenges that arise in real photometric stereo systems. Real acquisitions face additional difficulties including interreflections between surface regions, cast shadows that violate the single-light assumption, and non-Lambertian reflectance that breaks the linear image formation model.

The experiments assume global illumination with a single uniform light direction per image. Real systems often have near-field lighting effects where the illumination direction varies across the surface, requiring more sophisticated calibration. Self-occlusion and cast shadows, which create regions with no valid gradient information, are not modeled in these synthetic tests.

A Python Code Listings

This appendix contains the complete Python implementation with config, runner, surfaces, solvers, photometric stereo, and visualization modules.

A.1 Configuration (config.py)

```
# Grid resolution
NX, NY = 256, 256

# Domain extent
X_MIN, X_MAX = -2.0, 2.0
Y_MIN, Y_MAX = -2.0, 2.0

# Lighting configuration
DEFAULT_NUM_LIGHTS = 32
DEFAULT_ELEVATION_DEG = 45.0

# CG solver parameters
CG_TOL = 1e-10
CG_MAX_ITER = 5000
```

A.2 FFT Solver (solvers/fft_periodic.py)

```
import numpy as np

def solve_poisson_fft(f, dx, dy):
    """FFT-based Poisson solver with periodic BC."""
    Ny, Nx = f.shape

    # Build frequency grid
    kx = np.fft.fftfreq(Nx, d=dx) * 2 * np.pi
    ky = np.fft.fftfreq(Ny, d=dy) * 2 * np.pi
    KX, KY = np.meshgrid(kx, ky)

    # Laplacian in Fourier space
    denom = -(KX**2 + KY**2)
    denom[0, 0] = 1 # Avoid division by zero

    # Transform, solve, inverse
    F_hat = np.fft.fft2(f)
    Z_hat = F_hat / denom
    Z_hat[0, 0] = 0 # DC component

    z = np.real(np.fft.ifft2(Z_hat))
    return z - np.mean(z)
```

A.3 DCT Solver (solvers/dct_neumann.py)

```
from scipy.fftpack import dct, idct

def dct2(x):
    return dct(dct(x.T, norm='ortho').T, norm='ortho')
```

```

def idct2(x):
    return idct(idct(x.T, norm='ortho').T, norm='ortho')

def solve_poisson_dct_neumann(f, dx, dy):
    """DCT-based Poisson solver with Neumann BC."""
    Ny, Nx = f.shape
    f = f - np.mean(f) # Compatibility

    # Eigenvalues
    ii, jj = np.arange(Nx), np.arange(Ny)
    lam_x = -2/dx**2 * (1 - np.cos(np.pi * ii / Nx))
    lam_y = -2/dy**2 * (1 - np.cos(np.pi * jj / Ny))
    LX, LY = np.meshgrid(lam_x, lam_y)
    lam = LX + LY
    lam[0, 0] = 1

    Z_hat = dct2(f) / lam
    Z_hat[0, 0] = 0
    return idct2(Z_hat) - np.mean(idct2(Z_hat))

```

A.4 CG Solver (solvers/cg_iterative.py)

```

from scipy import sparse
from scipy.sparse.linalg import cg

def solve_poisson_cg(f, dx, dy, tol=1e-10):
    """CG Poisson solver with Dirichlet BC."""
    ny, nx = f.shape
    N = nx * ny

    cx, cy = 1/dx**2, 1/dy**2
    cc = -2 * (cx + cy)

    # Build sparse Laplacian
    rows, cols, vals = [], [], []
    for i in range(ny):
        for j in range(nx):
            k = i * nx + j
            if i == 0 or i == ny-1 or j == 0 or j == nx-1:
                rows.append(k); cols.append(k); vals.append(1.0)
            else:
                rows.append(k); cols.append(k); vals.append(cc)
                rows.append(k); cols.append(k-1); vals.append(cx)
                rows.append(k); cols.append(k+1); vals.append(cx)
                rows.append(k); cols.append(k-nx); vals.append(cy)
                rows.append(k); cols.append(k+nx); vals.append(cy)

    A = sparse.csr_matrix((vals, (rows, cols)), shape=(N, N))
    b = f.flatten()
    # Zero boundary RHS
    for i in range(ny):
        for j in range(nx):
            if i == 0 or i == ny-1 or j == 0 or j == nx-1:
                b[i*nx + j] = 0

    z, _ = cg(A, b, rtol=tol)
    return z.reshape((ny, nx)) - np.mean(z)

```


A.5 FD Solver (solvers/fd_dirichlet.py)

```

from scipy import sparse
from scipy.sparse.linalg import cg

def solve_poisson_fd_dirichlet(f, dx, dy):
    """Finite difference Poisson solver with Dirichlet BC."""
    Ny, Nx = f.shape
    N = Nx * Ny
    h2 = dx * dy

    # Build sparse Laplacian with 5-point stencil
    main_diag = -4.0 * np.ones(N)
    off_diag_1 = np.ones(N - 1)
    off_diag_Nx = np.ones(N - Nx)

    # Remove connections across row boundaries
    for i in range(1, Ny):
        off_diag_1[i * Nx - 1] = 0

    A = sparse.diags([main_diag, off_diag_1, off_diag_1,
                     off_diag_Nx, off_diag_Nx],
                     [0, 1, -1, Nx, -Nx], format='csr') / h2

    z_flat, _ = cg(A, f.flatten(), maxiter=2000, rtol=1e-8)
    return z_flat.reshape((Ny, Nx))

```

A.6 Visualization (visualization/)

```

# visualization/surfaces_3d.py
def save_3d_surface(X, Y, Z, filepath, title=''):
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')
    ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none')
    ax.set_title(title)
    plt.savefig(filepath, dpi=150, bbox_inches='tight')
    plt.close()

# visualization/heatmaps.py
def save_depth_heatmap(Z, filepath, title=''):
    plt.figure(figsize=(6, 5))
    plt.imshow(Z, cmap='viridis', origin='lower')
    plt.colorbar(label='Height')
    plt.title(title)
    plt.savefig(filepath, dpi=150, bbox_inches='tight')
    plt.close()

# visualization/histograms.py
def save_error_histogram(error, filepath, title=''):
    plt.figure(figsize=(6, 4))
    plt.hist(error.flatten(), bins=50, edgecolor='black')
    plt.xlabel('Error'); plt.ylabel('Count')
    plt.title(title)
    plt.savefig(filepath, dpi=150, bbox_inches='tight')
    plt.close()

# visualization/profiles.py

```

```
def save_profile_comparison(Z_true, Z_est, filepath):
    mid = Z_true.shape[0] // 2
    plt.figure(figsize=(8, 4))
    plt.plot(Z_true[mid, :], 'b-', label='True', linewidth=2)
    plt.plot(Z_est[mid, :], 'r--', label='Estimated', linewidth=2)
    plt.legend(); plt.xlabel('X'); plt.ylabel('Height')
    plt.savefig(filepath, dpi=150, bbox_inches='tight')
    plt.close()
```

A.7 Surface Generators (surfaces/)

```
def create_gaussian_surface(Nx=256, Ny=256, sigma=0.5):
    x, y = np.linspace(-2, 2, Nx), np.linspace(-2, 2, Ny)
    X, Y = np.meshgrid(x, y)
    Z = np.exp(-(X**2 + Y**2) / (2*sigma**2))
    return X, Y, Z, x[1]-x[0], y[1]-y[0]

def create_sphere_surface(Nx=256, Ny=256, radius=1.5):
    X, Y = np.meshgrid(np.linspace(-2, 2, Nx), np.linspace(-2, 2, Ny))
    r2 = X**2 + Y**2
    Z = np.where(r2 < radius**2, np.sqrt(radius**2 - r2), 0)
    return X, Y, Z, 4/(Nx-1), 4/(Ny-1)

def create_cone_surface(Nx=256, Ny=256, height=1.5):
    X, Y = np.meshgrid(np.linspace(-2, 2, Nx), np.linspace(-2, 2, Ny))
    Z = np.maximum(0, height - np.sqrt(X**2 + Y**2))
    return X, Y, Z, 4/(Nx-1), 4/(Ny-1)

def create_saddle_surface(Nx=256, Ny=256, scale=0.3):
    X, Y = np.meshgrid(np.linspace(-2, 2, Nx), np.linspace(-2, 2, Ny))
    Z = scale * (X**2 - Y**2)
    return X, Y, Z, 4/(Nx-1), 4/(Ny-1)

def create_sinusoid_surface(Nx=256, Ny=256, freq=2):
    X, Y = np.meshgrid(np.linspace(-2, 2, Nx), np.linspace(-2, 2, Ny))
    Z = 0.5 * np.sin(freq*np.pi*X) * np.sin(freq*np.pi*Y)
    return X, Y, Z, 4/(Nx-1), 4/(Ny-1)

def create_peaks_surface(Nx=256, Ny=256):
    X, Y = np.meshgrid(np.linspace(-3, 3, Nx), np.linspace(-3, 3, Ny))
    Z = 3*(1-X)**2*np.exp(-X**2-(Y+1)**2) \
        - 10*(X/5-X**3-Y**5)*np.exp(-X**2-Y**2) \
        - 1/3*np.exp(-(X+1)**2-Y**2)
    return X, Y, Z * 0.15, 6/(Nx-1), 6/(Ny-1)
```

A.8 Photometric Stereo Pipeline

```
# photometric/lighting.py
def make_rotating_lights(m=16, elevation_deg=45.0):
    """Generate m lights uniformly spaced in azimuth."""
    elevation = np.deg2rad(elevation_deg)
    azimuths = np.linspace(0, 2*np.pi, m, endpoint=False)
    lights = np.zeros((m, 3))
    lights[:, 0] = np.cos(azimuths) * np.sin(elevation)
    lights[:, 1] = np.sin(azimuths) * np.sin(elevation)
```

```

    lights[:, 2] = np.cos(elevation)
    return lights

# photometric/stereo.py
def photometric_stereo(images, lights):
    """Per-pixel least-squares PS: solve  $S @ g = I$ ."""
    m, Ny, Nx = images.shape
    I = images.reshape(m, -1)
    S_pinv = np.linalg.pinv(lights)
    G = S_pinv @ I
    norms = np.maximum(np.linalg.norm(G, axis=0, keepdims=True), 1e-10)
    N_flat = G / norms
    return N_flat.T.reshape(Ny, Nx, 3)

def gradients_from_normals(N_est):
    """Convert normals to gradients:  $p = -nx/nz$ ,  $q = -ny/nz$ ."""
    nz = np.where(np.abs(N_est[:, :, 2]) > 1e-6, N_est[:, :, 2], 1e-6)
    return -N_est[:, :, 0]/nz, -N_est[:, :, 1]/nz

# photometric/gradient.py
def compute_divergence(p, q, dx, dy):
    """Compute  $\text{div}(p, q) = dp/dx + dq/dy$ ."""
    dp_dx = np.gradient(p, dx, axis=1)
    dq_dy = np.gradient(q, dy, axis=0)
    return dp_dx + dq_dy

```

A.9 Main Runner (runner.py)

```

from experiments.exp_solver_compare import run_all_shapes_all_solvers

def main():
    results = run_all_shapes_all_solvers()
    print_results_table(results)
    save_results(results, 'solver_comparison_results.json')

if __name__ == "__main__":
    main()

```

B MATLAB Code Listings

This appendix contains the complete MATLAB implementation with config, runner, surfaces, solvers, photometric stereo, and visualization modules.

B.1 Configuration (config.m)

```
% config.m - Configuration parameters
NX = 128; NY = 128;
X_MIN = -2.0; X_MAX = 2.0; Y_MIN = -2.0; Y_MAX = 2.0;
DEFAULT_NUM_LIGHTS = 32;
DEFAULT_ELEVATION_DEG = 45.0;
CG_TOL = 1e-10; CG_MAX_ITER = 5000;
```

B.2 Main Runner (runner.m)

```
% runner.m - Main experiment orchestrator
addpath('surfaces','solvers','photometric','visualization');
run('config.m');
% Define surfaces and solvers, loop over combinations
for i = 1:length(surface_names)
    [X, Y, Z_true, dx, dy] = surfaces.(sname)();
    N = height_to_normals(Z_true, dx, dy);
    [p, q] = normals_to_gradients(N);
    f = compute_divergence(p, q, dx, dy);
    Z_est = solvers.(solver_name)(f, dx, dy);
    rmse = sqrt(mean((Z_true(:) - Z_est(:)).^2));
end
```

B.3 Surface Generators (surfaces/)

```
function [X,Y,Z,dx,dy] = create_gaussian_surface(Nx, Ny)
    [X,Y] = meshgrid(linspace(-2,2,Nx), linspace(-2,2,Ny));
    Z = exp(-(X.^2+Y.^2)/0.5); dx=4/(Nx-1); dy=4/(Ny-1);
end

function [X,Y,Z,dx,dy] = create_sphere_surface(Nx, Ny)
    [X,Y] = meshgrid(linspace(-2,2,Nx), linspace(-2,2,Ny));
    r2 = X.^2+Y.^2; R=1.5;
    Z = zeros(size(X)); Z(r2<R^2) = sqrt(R^2-r2(r2<R^2));
    dx=4/(Nx-1); dy=4/(Ny-1);
end

function [X,Y,Z,dx,dy] = create_cone_surface(Nx, Ny)
    [X,Y] = meshgrid(linspace(-2,2,Nx), linspace(-2,2,Ny));
    Z = max(0, 1.5 - sqrt(X.^2+Y.^2)); dx=4/(Nx-1); dy=4/(Ny-1);
end

function [X,Y,Z,dx,dy] = create_saddle_surface(Nx, Ny)
    [X,Y] = meshgrid(linspace(-2,2,Nx), linspace(-2,2,Ny));
    Z = 0.3*(X.^2-Y.^2); dx=4/(Nx-1); dy=4/(Ny-1);
end

function [X,Y,Z,dx,dy] = create_sinusoid_surface(Nx, Ny)
```

```

[X,Y] = meshgrid(linspace(-2,2,Nx), linspace(-2,2,Ny));
Z = 0.5*sin(2*pi*X).*sin(2*pi*Y); dx=4/(Nx-1); dy=4/(Ny-1);
end

function [X,Y,Z,dx,dy] = create_peaks_surface(Nx, Ny)
[X,Y] = meshgrid(linspace(-3,3,Nx), linspace(-3,3,Ny));
Z = 0.15*peaks(Nx); dx=6/(Nx-1); dy=6/(Ny-1);
end

```

B.4 FFT Solver (solve_poisson_fft.m)

```

function Z = solve_poisson_fft(f, dx, dy)
% FFT-based Poisson solver with periodic BC
[Ny, Nx] = size(f);

kx = (2*pi / (Nx*dx)) * [0:Nx/2-1, -Nx/2:-1];
ky = (2*pi / (Ny*dy)) * [0:Ny/2-1, -Ny/2:-1];
[KX, KY] = meshgrid(kx, ky);

denom = -(KX.^2 + KY.^2);
denom(1, 1) = 1;

F_hat = fft2(f);
Z_hat = F_hat ./ denom;
Z_hat(1, 1) = 0;

Z = real(ifft2(Z_hat));
Z = Z - mean(Z(:));
end

```

B.5 DCT Solver (solve_poisson_dct.m)

```

function Z = solve_poisson_dct(f, dx, dy)
% DCT-based Poisson solver with Neumann BC
[Ny, Nx] = size(f);
f = f - mean(f(:));

ii = 0:Nx-1; jj = 0:Ny-1;
lambda_x = -2/dx^2 * (1 - cos(pi * ii / Nx));
lambda_y = -2/dy^2 * (1 - cos(pi * jj / Ny));
[LX, LY] = meshgrid(lambda_x, lambda_y);
lambda = LX + LY;
lambda(1, 1) = 1;

F_hat = dct2(f);
Z_hat = F_hat ./ lambda;
Z_hat(1, 1) = 0;

Z = idct2(Z_hat);
Z = Z - mean(Z(:));
end

```

B.6 FD/CG Solver (solve_poisson_fd.m)

```

function Z = solve_poisson_fd(f, dx, dy)
% Finite difference solver with Dirichlet BC
[Ny, Nx] = size(f);
N = Nx * Ny;

cx = 1/dx^2; cy = 1/dy^2;
cc = -2 * (cx + cy);

% Build sparse Laplacian using spdiags
d0 = cc * ones(N, 1);
d1 = cx * ones(N, 1);
dNx = cy * ones(N, 1);

A = spdiags([dNx, d1, d0, d1, dNx], ...
            [-Nx, -1, 0, 1, Nx], N, N);

% Apply Dirichlet BC
b = f(:);
for i = 1:Ny
    for j = 1:Nx
        k = (i-1)*Nx + j;
        if i==1 || i==Ny || j==1 || j==Nx
            A(k,:) = 0; A(k,k) = 1; b(k) = 0;
        end
    end
end

Z = reshape(A \ b, [Ny, Nx]);
Z = Z - mean(Z(:));
end

```

B.7 Photometric Stereo Functions

```

function lights = make_rotating_lights(num_lights, elevation_deg)
% Generate light directions uniformly around azimuth
elev = deg2rad(elevation_deg);
azimuth = linspace(0, 2*pi, num_lights + 1);
azimuth = azimuth(1:end-1);

lights = zeros(num_lights, 3);
for i = 1:num_lights
    lights(i,:) = [cos(elev)*cos(azimuth(i)), ...
                  cos(elev)*sin(azimuth(i)), ...
                  sin(elev)];
end
end

function N = height_to_normals(Z, dx, dy)
% Compute normals from height field
[Zy, Zx] = gradient(Z, dx, dy);
norm_val = sqrt(Zx.^2 + Zy.^2 + 1);
N = cat(3, -Zx./norm_val, -Zy./norm_val, 1./norm_val);
end

function [p, q] = normals_to_gradients(N)
% Convert normals to gradients

```

```

    Nz = N(:,:,3); Nz(Nz < 1e-10) = 1e-10;
    p = -N(:,:,1) ./ Nz;
    q = -N(:,:,2) ./ Nz;
end

function f = compute_divergence(p, q, dx, dy)
% Compute divergence for Poisson RHS
    [~, dp_dx] = gradient(p, dx, dy);
    [dq_dy, ~] = gradient(q, dx, dy);
    f = dp_dx + dq_dy;
end

```

B.8 Visualization (visualization/save_surface_figures.m)

```

function save_surface_figures(X, Y, Z_true, Z_est, out_dir, shape, solver)
% Generate and save 7 visualization figures
    full_dir = fullfile(out_dir, shape, solver);
    if ~exist(full_dir, 'dir'), mkdir(full_dir); end

    err = Z_true - Z_est;
    rmse = sqrt(mean(err(:).^2));

    % 3D True Surface
    fig = figure('Visible','off');
    surf(X, Y, Z_true, 'EdgeColor','none'); colormap parula;
    saveas(fig, fullfile(full_dir, '3d_true.png')); close(fig);

    % 3D Estimated Surface
    fig = figure('Visible','off');
    surf(X, Y, Z_est, 'EdgeColor','none');
    title(sprintf('RMSE = %.4f', rmse));
    saveas(fig, fullfile(full_dir, '3d_est.png')); close(fig);

    % Depth maps, error map, profile, histogram (7 total)
end

```